DATAPROC SCENARIO BASED Q&A

BY - SHUBHAM WADEKAR

Copyright © Shubham Wadekar. All rights reserved.

This material is for personal use only. No part may be copied, shared, resold, or published without written permission. Unauthorized distribution is strictly prohibited and may result in action.

For permissions or licensing, contact: [shubham.p.wadekar@gmail.com]

Disclaimer: This content is for educational purposes. Accuracy is attempted but not guaranteed.

No job outcome is promised.

© Shubham Wa

1. Your pipeline requires only PySpark jobs on small datasets - would you use a single-node or standard cluster? Why?

I'd pick a single-node (primary-only) cluster for this use case because it's simpler and much cheaper. For small datasets the overhead of provisioning multiple nodes, shuffle traffic and YARN/container scheduling dominates runtime; a single-node Dataproc/Spark setup starts faster, has less operational surface, and usually finishes jobs quicker for lightweight workloads.

Second, single-node clusters remove network shuffle penalties and reduce inter-node latency - that's a win for tiny jobs where parallelism across machines doesn't help because the data fits comfortably in one machine's memory/CPU. You avoid paying for idle worker nodes and you don't need to tune executor/core ratios or worry about hot keys across machines.

Third, accept the tradeoffs explicitly: single node has limited fault tolerance (if the node dies the job fails) and limited horizontal scalability. That's fine for non-critical ad hoc jobs, development, unit tests, or scheduled small transforms, but not for production SLAs that require high availability or heavy parallelism.

Fourth, operationally I'd configure the node size to match the job (e.g., a higher-memory primary) and use GCS for durable staging/input so state isn't lost if you tear the cluster down. Keep jobs idempotent and small so you can re-run quickly if the single node dies.

Finally, if requirements change, design the pipeline so it's portable: write Spark jobs with sensible partitioning and avoid single-node hacks so you can switch to a standard multi-node cluster later with minimal code changes. That gives the best balance of cost, simplicity, and future-proofing for small PySpark workloads.

2. A team wants to process large production jobs with fault tolerance. How would you design the cluster type?

For large, production-grade jobs I'd use a standard multi-node cluster (Dataproc/YARN with Spark executors spread across workers) configured for fault tolerance and autoscaling. Multiple worker nodes give the horizontal parallelism you need for big datasets and let the cluster absorb node failures without aborting the entire job if you design for resilience.

Second, enable autoscaling and a mix of worker types to control cost and throughput: use a baseline of non-preemptible workers for stability and add preemptible workers for extra capacity during heavy stages. Combine this with an autoscaler policy that scales up for heavy shuffle phases and scales down when idle so you don't pay for capacity you don't need.

Third, harden Spark for production: enable checkpointing for long stateful jobs, configure external shuffle (or the managed shuffle service) to survive executor restarts, tune spark.speculation/dynamic allocation where helpful, and push intermediate outputs to durable storage (GCS/BigQuery) so you can recover or resume partial work. Use durable cluster metadata (metastore, job history) outside the ephemeral cluster if you tear clusters down after runs.

Fourth, consider high-availability masters or running critical control services in HA if your orchestration/latency SLAs demand it. Also bake in operational practices: strong monitoring (CPU, GC, shuffle IO), alerting, and automated retries for transient failures. Keep init actions and container images consistent (use Flex templates or initialization scripts) so any new node comes up ready.

Finally, secure and test the environment: lock down IAM to least privilege, use CMEK/KMS for sensitive data, run representative scale tests to size nodes and shuffle partitions, and codify the cluster template in IaC so the environment is reproducible. That combination -multi-node standard cluster, autoscaling, checkpointing/durable storage, and good ops -gives reliable, fault-tolerant processing for large production Spark jobs.

© Shubham Wadekar

3. You want jobs to run only when needed, with no idle cost. How would you use ephemeral clusters?

I'd treat clusters as cattle, not pets -spin them up **per job** and tear them down immediately when the job finishes. Practically that means automating cluster lifecycle in the job submission flow: the orchestration (Cloud Composer / Cloud Build / a CI job / Cloud Run) creates a small Dataproc cluster with the right machine types, submits the job, waits for completion, then deletes the cluster. That way you pay only for the run-time of the job and have zero idle worker cost between runs.

To make startup time acceptable, I'd optimize the cluster bootstrap: bake a custom Dataproc image or use initialization actions to pre-install dependencies and cache jars/wheels so jobs don't waste minutes installing packages. For very short jobs consider Dataproc Serverless (or managed serverless runners) if available -same no-idle-cost goal but with lower startup friction because the service manages provisioning for you.

Cost control and reliability go together: use small non-preemptible masters and add preemptible workers if you want cheaper temporary capacity during heavy stages; configure job retry and checkpointing (write intermediate results to GCS/BigQuery) so preemption does not force full reruns. Also parameterize cluster size and machine type so the orchestrator can choose right-sized resources based on the job's input size (use a quick sizing POC or a heuristic based on bytes to process).

Operationally, make the lifecycle idempotent and observable: maintain a job manifest that records cluster id, job id, input files/checksums, start/end times and exit status, and emit metrics (cost per job, start-up time, run-time). Expose logs and use structured tags (pipeline/version, commit SHA) so any transient failure is easy to correlate and re-run. Keep a backoff policy for retries that avoids repeatedly spinning up clusters during dependent-system outages.

Finally, bake reprocessing/runbook patterns: because you tear clusters down, keep raw inputs and intermediate outputs durable (GCS, BigQuery) so replays are safe, and offer a simple CLI or "run job" endpoint to let engineers start jobs on-demand. That gives the best of both worlds -no idle cost plus predictable, repeatable runs with safe recovery and auditability.

4. A client prefers submitting jobs via REST API, but developers are used to CLI. How would you support both?

I'd provide a single server-side submission layer (a small API service) that implements the canonical job-launch logic and exposes a clean REST contract the client can call. That service validates parameters, fills in environment defaults, enforces IAM/policy checks, kicks off the ephemeral cluster or Dataflow/Dataproc job, and returns job ids and a monitoring URL. Centralizing submission ensures security, consistent templates, and auditability.

For CLI-first developers, I'd ship a thin CLI wrapper that calls that same REST API -a small script or packaged tool (pip/npm/binary) so the devs keep their shell workflow (e.g., ./etl run --date 2025-09-01) but under the hood it uses the guarded API. This lets you evolve the server-side workflow without forcing everyone to learn new invocation details, and the CLI also supplies helpful defaults, templated parameter files, and local validation before submission.

If some teams insist on using the native cloud CLI, I'd still integrate: expose the server-side API endpoints as Cloud Build triggers or Cloud Run endpoints and provide Cloud CLI aliases or gcloud wrapper commands that either call the REST API or submit prebuilt templates (Dataproc job templates, Dataflow Flex template invocations). Documentation and examples -one-shot gcloud invocations, the wrapper CLI, and direct REST examples -let each developer use their preferred tool while using the same server-side logic.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

Security and IAM are important: authenticate API calls with short-lived credentials (Cloud IAM OAuth, Workload Identity) and require the service to impersonate the appropriate service account for job execution. Log all submissions (who, when, parameters) for audit and add server-side validation to prevent dangerous parameter combos. The CLI should support local credential forwarding so developers' actions are auditable under their identities.

Lastly, automate operability: add job-status endpoints, notifications (Slack/email) on completion/failure, and a "replay" endpoint that reads a manifest and restarts a job with the same inputs. Put the wrapper CLI and sample curl/Postman snippets in a developer README and CI templates so both REST and CLI workflows are first-class, consistent, and easy to adopt.

5. You're migrating Spark jobs from on-prem Hadoop. What would you change in job configuration for Dataproc?

Move from machine-centric tuning to cloud-friendly, elastic settings. On-prem you probably tuned for fixed node counts and local HDFS; on Dataproc you should parameterize executor counts/sizes and prefer autoscaling or ephemeral clusters. Replace hardcoded --num-executors fixed values with either dynamic allocation (spark.dynamicAllocation.enabled=true) or make executor counts a runtime parameter so you can right-size per job without rebuilding the job JAR. Use template parameters for cluster size and job options so CI/CD can change them per environment or data volume.

Tune memory and CPU for the cloud VM shapes you pick. Dataproc nodes have different vCPU/memory ratios than your old racks, so set spark.executor.memory, spark.executor.cores, and spark.driver.memory to match the instance type (leave headroom for OS and shuffle). Also set spark.yarn.executor.memoryOverhead (or spark.executor.memoryOverhead) to account for off-heap and python/native libs; the cloud environment often needs a slightly larger overhead than a tuned on-prem box if you use Python UDFs, Arrow, or JNI libraries.

Make IO and storage changes: replace HDFS paths with GCS URIs and use the GCS connector settings (fs.gs.system.bucket, spark.hadoop.fs.gs.requester.pays etc.) as needed. Prefer columnar, compressed formats (Parquet/Avro) on GCS to minimize egress and shuffle. For intermediate shuffle durability, rely on Dataproc-managed shuffle or configure spark.local.dir to point to local SSDs if available for heavy shuffles (or use the managed shuffle on Dataproc Serverless/Dataflow where appropriate).

Adjust fault-tolerance and preemptible usage: in the cloud you can use preemptible workers for cost savings, so configure Spark to tolerate preemptions (checkpointing, write intermediate outputs to GCS, and accept task restarts). Use spark.yarn.max.executor.failures and checkpoint intervals for long jobs. If you plan ephemeral clusters per job, make sure your job writes intermediate state to durable GCS (so restarts aren't catastrophic) and your job submission automates cluster creation/deletion.

Finally, modernize operational knobs: enable better logging and monitoring integrations (Stackdriver/Cloud Monitoring agents or Dataproc's integration), add init actions to install required libs, and bake the runtime environment into images to speed cluster bootstrap. Add CI tests for config matrix (small/medium/large datasets) so you validate the new cloud settings, and document the new sizing rules so operators can reproduce or tune cluster parameters without guessing.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

6. A Spark job is failing because of YARN resource contention. How would you fix it?

First, identify the contention pattern: look at YARN/NM logs and Spark UI to see whether executors are failing to allocate, heavy GC is blocking slots, or one stage creates large shuffle/executor memory pressure. Concrete metrics -container wait time, free memory per node, number of failed containers -tell you whether the issue is cluster-wide resource shortage, a skewed stage, or misconfigured executor sizing.

If cluster-wide shortage, increase cluster capacity or change node types. Add workers or choose machines with more memory/CPU, or allow preemptible workers to add capacity for heavy stages. Alternatively enable autoscaling (Dataproc autoscaling policies) so the cluster grows when demand spikes and shrinks afterward, avoiding permanent over-provisioning.

If contention is caused by per-executor oversizing, reduce spark.executor.cores and spark.executor.memory so each node can host more executors and YARN can place containers more flexibly. For example, if you used 8 cores per executor on 8-core nodes, switch to 2–4 cores per executor to reduce each container's footprint and improve parallel placement. Also tune spark.yarn.executor.memoryOverhead so JVM/NATIVE usage doesn't cause OOMs that make containers fail and requeue.

Address skew & shuffle hotspots: a single hot key or a heavy shuffle will monopolize slots and create contention. Detect hot keys (Spark UI / stage metrics) and apply salting, pre-aggregation, or map-side combine (CombineByKey/reduceByKey) to reduce the largest partitions. Repartition or coalesce before expensive operations to balance parallelism and avoid one task blowing out memory while others are idle.

Finally, use YARN/Spark scheduling controls: enable fair scheduler or queueing to prevent a single job/stage from starving others, set spark.speculation for straggler mitigation, and limit concurrency via spark.yarn.am.memory or queue configs. Add retries and backoff for transient allocation failures, and instrument monitoring alerts for container failure rates and queue wait times so you can react before jobs crash.

7. A Hive query must run on Dataproc using SparkSQL and share schema definitions. How would you set this up?

Start by centralizing the Hive Metastore so every Dataproc cluster can reuse the same table/catalog metadata. The cleanest way on GCP is to run a managed metastore (Dataproc Metastore / Cloud SQL-backed Hive metastore) or a single, highly-available HMS (Cloud SQL + MySQL/Postgres) that you point every cluster at. This gives you one canonical place for table definitions, partitions, SerDe configs and ACLs so SparkSQL on any cluster sees the same schema.

When you create your Dataproc clusters, configure them to use that central metastore by deploying a hive-site.xml that contains the metastore JDBC URL, credentials (use Secret Manager or instance IAM), and the hive.metastore.warehouse.dir pointing to a shared GCS warehouse (e.g. gs://my-data-warehouse/warehouse/). Put that hive-site.xml into the cluster image or apply it at startup via an init action so Spark picks it up automatically. Ensure spark.sql.catalogImplementation is set to hive (Spark with Hive support) so spark.sql("select * from db.table") consults the shared metastore.

Use Hive-compatible storage formats and partitioning (Parquet/ORC + partitioned directories) and keep the physical data in GCS. That avoids HDFS dependence and makes the data available to all clusters. When creating tables, prefer external tables with explicit locations and add partitioning and clustering practices in your table DDL so both Hive and Spark queries have predictable performance and avoid accidental rewrites of data.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

Manage schema evolution and governance: version your DDL changes via schema migrations (SQL files in source control), require PRs for schema changes, and test migrations in a staging metastore before promoting. Capture metadata and access policies in Data Catalog or your governance tool so auditors and downstream consumers can trace changes. Also script safe backfills (ALTER TABLE ADD COLUMN AS NULLABLE then backfill with a scheduled job) to avoid breaking running Spark jobs.

Operationalize and validate: build CI that runs a small SparkSQL job against the staging metastore to verify queries, and provide runbooks for adding new clusters (how to mount hive-site.xml, required IAM roles like roles/storage.objectViewer on the warehouse bucket, and service account KMS permissions if CMEK is used). Finally, monitor metastore health, GCS latency, and table partition counts so you catch catalog or IO problems early -that keeps Hive and Spark working consistently across Dataproc clusters.

8. A project requires specific Spark version compatibility. How would you configure Dataproc?

Pin a Dataproc image version that includes the Spark release you need. Dataproc image families are versioned (for example Dataproc 2.x/1.x lines) and each image packages a particular Spark/Hadoop stack. Choose the image at cluster-creation time (via --image-version) so every cluster runs the exact Spark binary the job expects -this is the simplest, supported, and reproducible approach.

If the exact Spark minor/patch is not available in a managed Dataproc image, build a custom Dataproc image or use initialization actions to install the required Spark distribution on cluster startup. Dataproc supports creating custom images (or using containerized job execution) where you bake Spark + custom jars into the image; then reference that image in your IaC so clusters are identical and deployments are repeatable. Store the image definition and build in CI so upgrades are controlled and reviewed.

For even stronger reproducibility or if you need native libraries and Python/C++ deps, package the job runtime in a container and submit as a containerized job (Dataproc supports containerized workloads) or use Flex templates for Dataflow-like jobs. Container images let you pin not only Spark but also the exact Python, library versions, and OS packages. This is especially useful for machine-learning pipelines or UDFs that depend on native libs.

Don't forget the ecosystem compatibility matrix: match the Spark version with compatible Hadoop/GCS connector versions, Hive Metastore client and any third-party connectors (JDBC, Kafka, BigQuery connectors). Pin those dependency versions in your init actions or image and test the whole stack in a staging environment before moving to production -mismatched connector versions are a very common source of runtime failure.

Finally, automate version management: keep image/version parameters in your deployment configs (Terraform/Cloud Build), run automated integration tests that validate a subset of your jobs against the pinned Spark image, and have a documented upgrade path (build new image, run smoke tests, promote). That gives you deterministic, auditable Spark compatibility while allowing controlled, safe upgrades when the business is ready.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

9. You need a daily ETL pipeline (GCS → Dataproc → BigQuery). How would you design it?

Start with a clear landing/contract in GCS: producers write into a well-known prefix gs://bucket/source/YYYY/MM/DD/ and they atomically signal completeness (either by writing a .ready marker or via GCS Pub/Sub notifications). Keep raw files immutable and archived -never transform them in place -so you always have a replayable source of truth. Prefer columnar formats (Parquet/Avro) if producers can, otherwise use newline JSON/CSV but plan a conversion step early in the pipeline.

Orchestrate with Cloud Composer (Airflow) or Cloud Scheduler + Cloud Build: every day the orchestrator determines the partition(s) to process, checks for the .ready marker (or consults a manifest), and creates an ephemeral Dataproc cluster tuned to the job (right machine types, number of workers, preemptible workers for cost where acceptable). Ephemeral clusters avoid idle cost and can be pre-warmed with a custom image or init action to speed bootstrap (Spark jars, Python wheels, connectors).

On Dataproc run a Spark job that: (1) reads only the target partition(s) from GCS, (2) performs validation / cleansing / enrichment (with per-record error routing to a dead-letter GCS path), (3) writes cleaned data to a partitioned staging location -either partitioned Parquet files on GCS or a partitioned staging table in BigQuery. Keep transforms idempotent and deterministic and add audit columns (source_file, ingestion_ts, batch_id) to every row to trace provenance.

Finalize via an atomic handoff: if you wrote Parquet to GCS, submit a BigQuery load job for that partition (cheaper, faster than streaming for bulk). If you wrote into a staging BigQuery table, run a MERGE from staging to production to dedupe and apply business rules. Use partition decorators or date-partitioned tables so only the target partition gets touched and the run is re-runnable. After success update a processing manifest (BigQuery table) marking the partition/files as processed (include checksum/generation), then delete or archive the staging artifacts and tear down the cluster.

Instrument everything: metrics for file counts, row counts (source vs landed), DLQ counts, cluster startup time, and job duration. Add alerts on mismatches (row delta threshold), DLQ spikes, or cluster failures. Keep CI for Spark job unit tests, schema/contract checks, and a staged shadow run before cutover. Finally, secure service accounts with least privilege (GCS/BigQuery/KMS roles), and ensure CMEK, VPC-SC or other compliance settings are applied per your org rules.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

10. Incremental loads are required -how would you process only new partitions each run?

Use deterministic partitioning and a processed-manifest as the source of truth. Producers must write into date/partitioned paths (e.g., .../date=YYYY-MM-DD/). The pipeline queries a manifest table (a small BigQuery table you maintain) that records every processed partition/file with metadata (filename, GCS generation, checksum, processed_ts, batch_id). At the start of each run the orchestrator lists candidate partitions (or reacts to Pub/Sub notifications / .ready markers) and selects only those not present in the manifest (or whose generation/checksum changed) -that difference set is the incremental work list.

Prefer a marker-based pattern to avoid expensive full listings: producers write a .ready file into the partition when upload completes. The scheduler only attempts partitions with a ready file. After a successful load the pipeline writes an entry into the manifest in a single transaction (BigQuery insert) that includes partition, files, checksums, and status=SUCCESS. Use the manifest entry as a lock/lease (insert with unique key) to prevent concurrent runs from processing the same partition. For more complex coordination you can use Firestore or Datastore for lightweight locks.

Make the ingestion idempotent: write to a staging partition first, validate, then MERGE into production keyed by primary key + ingestion batch/version so replays (or reruns if a previous run failed mid-way) do not create duplicates. If a partition is partially processed, have the run mark status=FAILED and include error details; the operator can fix and re-run, and the pipeline logic should detect FAILED entries and allow safe retries after clearing or reconciling.

Support backfills and repairs by allowing parameterized runs (start_date, end_date, force=true) that ignore the manifest or selectively delete manifest entries for affected partitions, then reprocess them. Keep raw files immutable so reprocessing is deterministic. Also record checksums and row counts in the manifest so automated parity checks can detect silent data drift and trigger alerts or automated re-runs.

Operationalize monitoring and governance: alert if expected partitions (based on business calendar) do not appear, if manifests show repeated failures for the same partition, or if the manifest ages (oldest unprocessed .ready) exceed SLA. Document the manifest schema and runbooks for manual replay, and expose a lightweight UI or SQL view for operators to see what partitions have been processed and what remains. This combination -deterministic partitioning, ready markers, a manifest with locks, idempotent staging+MERGE -gives safe, efficient incremental ingestion.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

11. You need SQL-based transformations instead of writing PySpark. Would you use Hive or Presto? Why?

If your workload is primarily long-running ETL jobs that need strong integration with the Hive metastore, transactional write semantics, or heavy batch joins over very large datasets, I'd favour a Hive/Spark-SQL style solution (Hive on Tez or Spark SQL on Dataproc). Hive/SparkSQL is a natural fit for ETL because it supports table DDL, partitioned/writable tables, ACID-ish patterns (when using compatible storage/formats), bulk inserts/merges, and mature optimizations for large, disk-heavy joins. It plays nicely with existing Hive metadata and with large-scale job orchestration (Airflow/Composer → Dataproc), and it's easier to run deterministic, repeatable scheduled jobs that produce production tables or partitioned exports to BigQuery/GCS.

If your priority is interactive, low-latency, ad-hoc analytics and many small queries from BI users -or if you need a federated SQL engine that can query many connectors (S3/GCS, MySQL, BigQuery, Kafka) with low latency -Presto (or Trino) is usually the better choice. Presto/Trino is engineered for speed on interactive queries, uses ANSI SQL, has very fast response for selective scans, and supports high concurrency for BI tools. It's great when analysts want immediate feedback and you want to avoid the overhead of spinning up heavy batch clusters for every query.

Put simply: pick Hive/SparkSQL for heavy-duty batch ETL, deterministic transforms, large-scale joins and upserts; pick Presto/Trino for interactive analytics, federated queries, and BI workloads that require low-latency response. Both can coexist in the same ecosystem - Hive/Spark can produce the nightly canonical datasets and Presto can power dashboards over those curated datasets. Choose based on the primary consumer and SLAs rather than personal preference.

Operational considerations matter: if you already have a Hive metastore and lots of existing Hive DDL, migrating to SparkSQL preserves that investment. If you want an analyst-friendly engine with quick time-to-insight and many connectors, invest in Presto and put the curated tables where Presto reads them efficiently (partitioned Parquet on GCS, for example). Also factor in maintenance: Dataproc/Spark jobs require cluster management (or serverless Dataproc), while Presto clusters require tuning for concurrency and memory, and a coordinator/worker architecture.

Finally, I'd recommend a short POC: implement the same representative transformation in both engines, measure end-to-end runtime, concurrency behavior, cost, and developer velocity. Use those concrete numbers plus stakeholder preferences (batch vs interactive, required write semantics) to decide -that keeps the decision evidence-based and reduces second-guessing after cutover.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

12. A workflow must combine Dataflow streaming with Dataproc batch. How would you architect this hybrid pipeline?

Design the system so streaming and batch are separate, composable layers that share the same canonical data model and storage. In practice I start with Pub/Sub → Dataflow for real-time ingestion/transform: Dataflow validates, enriches, and writes two outputs -(a) low-latency results to BigQuery (or a low-latency sink) for dashboards/alerts, and (b) an immutable, partitioned raw/archive in GCS (or a changelog table in BigQuery) that acts as the single source of truth for batch reprocessing. That archive is critical: it lets Dataproc batch jobs re-run deterministic transforms for full correctness, backfills, or reprocessing after logic changes.

For the batch side use Dataproc to run cost-efficient, heavy transformations over historical windows: scheduled Dataproc jobs read the partitioned archive (Parquet/Avro on GCS), perform large joins/aggregations the streaming layer avoids or defers, and write canonical, partitioned analytics tables into BigQuery (or overwrite partitions). Keep the transform logic shared as much as possible -e.g., implement common transformation SQL or modular libraries so streaming and batch produce identical output schemas. That reduces drift between real-time and authoritative views.

Make the join/commit semantics idempotent and replayable: streaming emits best-effort/upserts with insertId or a staging table plus frequent MERGE, while Dataproc batch performs authoritative upserts or partition swaps. Use provenance/audit columns (source_batch_id, commit_ts, pipeline_version) and a manifest table to record which partitions/files were processed so replays are safe. Also provide a reconciliation job that compares streaming-derived metrics vs batch final metrics and raises alerts when divergence exceeds tolerance.

Operationally enforce consistency, security and scalability: run Dataflow and Dataproc within the same VPC/VPC-SC perimeter as needed, give least-privilege service accounts access only to required buckets/datasets, and use CMEK if required. Instrument both layers with unified metrics (watermark lag, backlog, job duration, DLQ counts) and a dashboard that shows real-time health plus last-successful-batch time so operators can spot when batch lags behind streaming. Schedule the Dataproc batch to run after expected ingestion windows and include automated retries and a manual reprocess playbook.

Finally, plan for evolution: keep transforms modular (shared code or SQL views), store schema contracts in a registry, and run CI to validate parity between streaming and batch outputs on sample data. This hybrid architecture gives you sub-second insights where needed and authoritative, lower-cost aggregations for analytics -the best of both worlds while keeping correctness auditable and reprocesses simple.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

13. Your team is debating between HDFS and GCS for storage. Which would you choose for Dataproc and why?

I'd choose GCS as the canonical storage layer for Dataproc. In the cloud the best-practice is to treat compute (Dataproc clusters) as ephemeral and storage as durable and independent -GCS is designed for that pattern: durable, globally available (or regionally scoped), and integrated with GCP IAM, KMS, lifecycle rules and logging. Putting your data in GCS lets you spin up and tear down Dataproc clusters without worrying about moving data in/out of HDFS, which simplifies operations and lowers cost because you don't pay for idle HDFS storage on long-running VMs.

Operationally, GCS removes a lot of operational friction: no need to manage HDFS NameNode HA, no manual replication tuning, and simpler backup/restore and retention via lifecycle rules. It also plays very well with all other GCP services (BigQuery load jobs, Dataflow, Dataproc, ML tooling), so a single canonical dataset stored in Parquet/Avro on GCS can be consumed by multiple engines without copy jobs. That typically reduces data duplication and ETL complexity compared to keeping data in an on-cluster HDFS cluster.

From a performance perspective, Dataproc's GCS connector is optimized for cloud patterns; for very heavy shuffle or when you need extremely low-latency local reads/writes, ephemeral local SSDs or the Dataproc shuffle service can be used during the job -but those are compute-local optimizations, not reasons to keep long-term data in HDFS. In short: use GCS for durable storage and leverage local disks/shuffle services only for intermediate, high-throughput phases of jobs.

There are a few caveats you should plan for: design your job IO for object storage (avoid many tiny file writes without compaction), partition and file-size tune (aim for hundreds of MB per file for analytics), and ensure you use the right consistency/visibility semantics (manifest files or _SUCCESS markers for consumers). Also secure buckets with least-privilege IAM, optionally enable CMEK if required, and use VPC/S private access patterns for sensitive data.

Finally, if you have a strict on-prem legacy requirement (e.g., existing HDFS-only tooling that cannot be changed), you can run HDFS on Dataproc nodes or stage data via connectors temporarily, but for a cloud-native, scalable, low-ops architecture, GCS is the recommended default.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

14. You need persistent metadata across clusters. How would you configure an external Hive Metastore?

Give the metastore a single authoritative endpoint that every cluster points to -either a managed Dataproc Metastore service or a self-hosted Hive Metastore backed by Cloud SQL. I prefer Dataproc Metastore (managed) when possible because it removes the operational burden: Google manages the service, you get HA, and Dataproc clusters can be configured to use that metastore at creation time. If your compliance or architecture forces self-hosting, run the metastore backed by a Cloud SQL instance (MySQL/Postgres) in a highly-available configuration.

Concretely, configure every Dataproc cluster to use the external metastore by supplying a hive-site.xml (or using Dataproc's --metadata/--properties flags) that points javax.jdo.option.ConnectionURL to your Cloud SQL instance, and include credentials securely (Secret Manager or Workload Identity) for the metastore user. Also set hive.metastore.warehouse.dir to a shared GCS warehouse path (for example gs://my-data-warehouse/warehouse/) so all clusters and engines agree on table locations; use external tables (explicit locations) where possible to keep data ownership explicit.

Network and IAM matter: put the metastore (Cloud SQL or Dataproc Metastore) and Dataproc clusters in the same VPC/region or use Private Services Connect so clusters can reach the metastore without public egress. Ensure the service account Dataproc uses has the right roles for accessing the metastore and the GCS warehouse (roles/storage.objectViewer/Creator scoped to the buckets). For Cloud SQL-backed metastore, use Cloud SQL Proxy or VPC peering and secure DB credentials, rotate them and log access for audits.

Plan for schema/versioning and HA: back up the metastore DB regularly, snapshot Cloud SQL, and maintain a migration process (DDL scripts) tracked in source control so schema changes are reviewed and reversible. If you need high availability and zero-downtime cluster launches, use Dataproc Metastore or Cloud SQL HA (read replicas and failover) so new clusters can always obtain table/partition metadata.

Finally, operationalize governance: integrate the metastore with Data Catalog for richer metadata and lineage, enforce a schema-change process (PRs, CI tests that validate DDL against sample data), and expose a small admin runbook for adding/removing clusters or recovering when metadata drift or partition inconsistencies appear. With a centrally managed metastore and shared GCS warehouse, all your Dataproc clusters will have consistent, persistent metadata and you avoid the classic headaches of per-cluster HDFS/Hive silos.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

15. Your job performance is slow because of too many small files in GCS. How would you optimize this?

Start by fixing the producer: buffer records and write larger files (flush after a size or time threshold) instead of emitting per-record objects. If you control the upstream writers, change them to write into a staging path and then atomically rename/move a single larger file into the ingest folder when complete -that prevents consumers from seeing lots of tiny, incomplete files and reduces list/read overhead.

If you can't change producers, add a compaction step before heavy processing. Run a short Dataflow/Dataproc job that reads many small files (by time window or prefix), coalesces the records and writes out larger Parquet/Avro files (~100MB–1GB target files depending on query patterns). Schedule compaction periodically (hourly/daily) so downstream ETL jobs read a compacted set rather than thousands of tiny objects.

Tune the Spark/Dataflow read configuration so the runtime doesn't spawn a task per tiny file. For Spark on Dataproc, control the number of input partitions with spark.read.option("maxPartitionBytes", ...) or explicitly repartition/coalesce after reading small files. For Beam/Dataflow, batch small reads before downstream transforms. Also prefer columnar formats (Parquet/Avro) for the compacted outputs -they compress well and enable predicate pushdown so downstream scans are cheaper.

Manage lifecycle and operational visibility: maintain a manifest of processed files (filename, generation, checksum) so you don't accidentally reprocess or double-compact, and emit metrics for average file size and files-per-minute. Add alerts when average file size drops below a threshold so you can catch regressions early rather than when the pipeline is already slow.

Finally, optimize consumers too: tune shuffle partitions and parallelism so a reasonable number of tasks handle the compacted files, and avoid tiny output shards when writing results (use coalesce/repartition when writing). Document the file-size targets and compaction cadence in the developer guide so future producers and pipelines follow the same best-practices and the small-file problem doesn't reappear.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

16. A Spark query is too slow on large tables. How would you use partitioning and bucketing to improve it?

Partition your tables on a high-cardinality column that's commonly used in filters (date, country, customer_id prefix). Partitioning enables partition pruning so Spark reads only the relevant directories instead of the whole dataset -that typically cuts scanned data dramatically for selective queries. Keep partition granularity sensible (daily/hourly for time-series; avoid millions of tiny partitions).

Use bucketing when you repeatedly join or aggregate on a particular key. Bucketing (writing data into N buckets by hash of the join key) lets Spark perform bucketed joins and avoids a full shuffle when both sides are bucketed with the same bucket count and key. For aggregations, pre-bucketing and sorting by the aggregation key speeds up group-by and reduces shuffle cost. Ensure the bucket count is tuned to cluster parallelism -too few buckets create hotspots, too many recreate small-file problems.

Combine partitioning + bucketing: partition by a coarse filter (e.g., date) and bucket within each partition by the join key. That gives fast pruning plus shuffle-reduced joins. When writing the data, use df.write.partitionBy(...).bucketBy(...).sortBy(...) (or the equivalent Hive/Spark table DDL) so downstream queries can leverage both optimizations. Be mindful that Hive-style bucketing creates files per bucket per partition -you must compact if they're tiny.

On the query side, ensure your SQL uses the partition column in WHERE clauses (so pruning can occur) and that joins reference the same key used for bucketing. Tune Spark's execution parameters too: set spark.sql.shuffle.partitions to a value appropriate for your data size and cluster, and consider adaptive query execution (AQE) if available to rebalance skew and coalesce shuffle partitions at runtime.

Finally, monitor and iterate: measure bytes read and shuffle bytes before/after changes, test with representative workloads, and watch for skewed buckets or hot partitions. If you still have hot keys, consider salting or special-case handling (pre-aggregation or separate stores) for those heavy hitters. With thoughtful partitioning and bucketing your queries will scan far less data and shuffle much less, which is where most Spark slowdowns come from.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

17. A downstream BigQuery system needs efficient analytics. Which file format would you choose in Dataproc?

I'd use Parquet as the primary file format when preparing data in Dataproc for BigQuery analytics. Parquet is a columnar storage format that compresses very well and allows efficient predicate pushdown. That means if BigQuery queries filter on a subset of columns or need aggregates, it only reads the relevant column chunks instead of scanning the whole row. This drastically reduces scan costs and improves query performance.

Parquet also handles nested and complex data structures better than row-based formats like CSV or JSON. Since BigQuery supports nested/repeated fields, keeping that structure in Parquet means the analytics system can query hierarchical data natively instead of flattening it prematurely. It's also more compact on disk, so GCS storage costs are lower and shuffle/IO overhead in Spark is reduced.

Another advantage is interoperability: Spark on Dataproc writes Parquet efficiently, BigQuery has first-class support for ingesting Parquet directly (either load jobs or external tables over GCS), and many other systems (Presto, Hive, ML frameworks) consume Parquet easily. That makes it a good default for a multi-engine ecosystem.

I'd only use Avro for BigQuery if schema evolution with many changing fields is the top concern or if I need row-based streaming ingestion. For analytics-focused workloads where queries scan large tables and filter by columns, Parquet is almost always the better choice.

So the design would be: Dataproc Spark jobs read/clean/transform, write partitioned Parquet to GCS (optimized file sizes ~256MB–1GB per file), and then BigQuery either directly queries via external tables or runs fast partitioned load jobs. This pipeline minimizes both storage cost and query latency in the downstream BigQuery system.

18. You must support schema evolution in your dataset. Would you use Avro or Parquet? Why?

For schema evolution, I'd choose Avro over Parquet. Avro was designed with schema evolution as a core feature -the schema is stored in the file header, and it's very good at handling forward and backward compatibility. If a producer adds new fields, old readers can ignore them; if fields are removed, Avro can substitute defaults. That makes it easy to evolve your dataset gradually without breaking existing consumers.

Parquet does have limited schema evolution support (adding new nullable columns, reordering), but it's stricter. For example, renaming fields or changing types is more problematic in Parquet. Avro's strong schema evolution story makes it the format of choice if your pipeline must handle frequent changes to the dataset schema, especially if multiple producers/consumers evolve at different times.

Another point is usage patterns: Avro is row-based, so it's efficient for write-heavy pipelines and good for streaming/transactional-style ingestion where you want to append evolving events. That matches well with ingestion layers (e.g., IoT events, CDC, logs). Once stabilized, you can still convert Avro to Parquet downstream for analytics.

Operationally, I'd also combine Avro with a schema registry (like Confluent Schema Registry or Data Catalog with JSON schemas) so that producers and consumers agree on schema versions. This avoids mismatches and provides governance/audit for schema evolution.

So my recommendation would be: use Avro if schema evolution is the key requirement, especially for ingestion and long-term raw archives. Then, for analytics efficiency, you can transform Avro → Parquet in Dataproc once the schema stabilizes. This pattern gives the flexibility of Avro for evolution and the efficiency of Parquet for downstream analytics.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

19. A pipeline is too slow due to large JSON files. How would you improve performance?

Start by stopping JSON from being the analytic format -convert the large JSON files as early as possible into a columnar, typed format (Parquet or Avro) using a one-time Dataproc/Dataflow job. Columnar formats drastically reduce I/O for analytic queries because they only read required columns, compress better, and enable predicate pushdown; converting once up-front turns every downstream job from "scan everything and parse text" into a fast columnar read. Keep the original JSON archive for replays, but run your daily ETL from Parquet.

Second, enforce a clear schema and use schema-aware readers. Parse JSON into a typed schema (Avro/Beam Row/Spark DataFrame schema) instead of inferring on the fly for every job. Schema-aware readers avoid repeated expensive parsing, let the engine optimize memory layout, and make casts/errors explicit (route bad rows to a DLQ). If producers change fields often, capture unknown fields into an extensions column so the main schema stays stable.

Third, partition and compact the converted files. Partition the Parquet output by a commonly-filtered column (date, region) to enable partition pruning, and produce target file sizes (hundreds of MB) to avoid many small-file overheads. If JSON files are huge, split them into logical shards during the conversion so the read parallelism can scale across many tasks; do the inverse if you're dealing with many tiny files -compact them.

Fourth, tune your compute and parsing strategy. Use a distributed engine (Dataproc/Spark or Dataflow) to parse JSON in parallel, prefer streaming/batched parsing libraries that use efficient binary codecs, batch external calls, and avoid expensive per-record external lookups in the hot path. Also tune executor memory/cores and number of partitions so parsing work is balanced (avoid very large partitions that provoke long GC pauses).

Finally, add operational safeguards: profile end-to-end (bytes read, parse time, CPU), add early validation to catch malformed JSON cheaply, and automate the conversion step in your pipeline (template/Flex job) so it runs whenever new JSON arrives. Measure the before/after gains -you'll usually see order-of-magnitude improvements simply by converting text JSON to a typed, columnar format and partitioning it for analytics.

20. You're asked to compress intermediate data -which codec would you choose and why?

For most intermediate/analytic pipelines I'd pick **Snappy** as the default codec. Snappy offers a very good balance of compression ratio and extremely fast (low-CPU) compress/decompress performance, which minimizes CPU overhead on workers and keeps I/O and shuffle throughput high. It's widely supported (Parquet, Avro, ORC, Spark, BigQuery) so interoperability is simple and you don't risk incompatibilities when moving data between systems.

If your primary goal is space savings and you can accept higher CPU cost, use Zstandard (ZSTD) at a tuned compression level. ZSTD typically gives much better compression than Snappy while still being faster than gzip/deflate for decompression; it's a great middle ground for archival or for pipelines where network/IO is the real bottleneck and you can afford more CPU. Test a few compression levels -often a medium setting gives most of the benefit with reasonable CPU.

Avoid gzip/deflate for hot intermediate data unless you absolutely need its compression ratio and you can afford the CPU/time -gzip slows job runtimes because decompression is expensive and not easily parallelized in many readers. Gzip is fine for long-term archival where decompress-once cost is acceptable, but not for frequent shuffle or interactive analytic reads.

Finally, be pragmatic and benchmark. Compression behavior depends on data characteristics (column cardinality, repetitiveness) and your cluster's CPU vs IO profile. Run a small experiment comparing Snappy vs ZSTD vs gzip on a representative dataset and measure end-to-end job time, network bytes transferred, and storage cost. Pick the codec that minimizes

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

total cost-of-processing for your workload; in practice that's usually Snappy for fast pipelines and ZSTD for storage-sensitive, less-latency-critical paths.

21. A job is failing due to driver out-of-memory errors. How would you tune Spark configs?

First, I'd stop guessing and ensure the driver actually needs more heap versus there being a real bug that accumulates huge objects. I run a representative job with monitoring (Spark UI, logs, and a heapdump if needed) to see what the driver is holding: large broadcast variables, collected collect() calls, large accumulators, or excessive cached metadata. If your code is calling collect(), toLocalIterator() or keeping whole datasets on the driver, fix the code first - move aggregation to executors, write to GCS/BigQuery, or sample instead of collecting everything.

If the driver legitimately needs more memory (e.g., large broadcast state, driver-side joins, or lots of task metadata for many partitions), increase the driver heap and its overhead. Practical config knobs are spark.driver.memory (heap) and spark.driver.memoryOverhead (off-heap/native overhead). As a starting heuristic, increase spark.driver.memory to match data needs (e.g., from 4g → 16g) and set spark.driver.memoryOverhead to 10–20% of the heap if you use native libs or Python/Arrow. Also pick a machine type with more vCPUs and memory for the driver node if you run on Dataproc/EMR.

Tune JVM and serialization for the driver workload: enable Kryo (spark.serializer=org.apache.spark.serializer.KryoSerializer) and register commonly used classes to reduce serialized size; raise spark.kryoserializer.buffer.max if you see buffer overflow errors. If your driver does a lot of metadata or Spark SQL planning, increasing spark.sql.broadcastTimeout or reducing spark.sql.autoBroadcastJoinThreshold can prevent the driver from holding huge broadcasted relations unexpectedly.

If you rely on large broadcast variables that must live on the driver, consider turning them into broadcasted Broadcast objects fault-tolerantly (i.e., use sc.broadcast() but ensure the broadcast size is reasonable), or better: store large side-data in an external store (Bigtable, Redis, Memcached) and do lookups from executors. Also consider enabling off-heap memory for Spark (spark.memory.offHeap.enabled=true and spark.memory.offHeap.size) if native libraries require it -but prefer increasing heap and fixing code before moving to off-heap complexity.

Finally, make the pipeline resilient: add checkpoints for long-running jobs, avoid very large numbers of tiny partitions (which blows up driver task metadata), and enable dynamic allocation to balance executors vs driver load. Document the memory changes and add a CI smoke test that runs a smaller but representative job so future changes don't reintroduce driver memory bloat.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

22. A Spark job is slow because of too many shuffle operations. How would you optimize it?

Start by profiling where shuffles happen: use the Spark UI to identify stages with huge shuffle read/write bytes and long shuffle times. Once you know the painful stages, try to reduce the amount of data shuffled -the most effective levers are pushdowns, early projections, and map-side aggregation. Remove unnecessary columns early (select only needed columns) and use reduceByKey/aggregateByKey/CombinePerKey instead of groupByKey so combiners reduce data before the expensive network shuffle.

Second, reduce the number of shuffles by fusing operations where possible and avoiding needless repartitions. Where a join causes a shuffle, consider broadcast joins (broadcast(smallDf)) if one side is small (spark.sql.autoBroadcastJoinThreshold controls automatic behavior). If both sides are large, ensure both are partitioned on the join key (bucketed or co-partitioned) so you can avoid a full shuffle or use shuffleHash/sortMerge appropriately; bucketing or repartitioning once and reusing that partitioning across multiple joins saves repeated shuffle cost.

Third, tune shuffle parallelism and adaptive execution. Set spark.sql.shuffle.partitions (or spark.default.parallelism) to a value proportional to your cluster (not the default 200 blindly) - too many partitions create overhead, too few cause long-running tasks. If your Spark version supports it, enable Adaptive Query Execution (AQE) which can coalesce shuffle partitions and handle skew at runtime (spark.sql.adaptive.enabled=true). AQE often gives big improvements with little code change.

Fourth, handle skew explicitly: detect heavy keys (top-key statistics) and apply salting for those keys -split a hot key into N shards during the map phase, aggregate in parallel, and then un-salt to combine partial results. For joins with skewed keys, pre-aggregate or isolate heavy keys into a special path processed separately (or use a targeted broadcast for those keys). Skew is a frequent cause of one or two tasks dominating stage time.

Finally, optimize IO and formats to shrink shuffle data: use compact columnar formats (Parquet) for intermediate storage, enable spark.sql.shuffle.partitions tuning, compress shuffle files with a fast codec (Snappy), and increase spark.shuffle.file.buffer / spark.reducer.maxSizeInFlight if network is limiting. Also ensure executor memory and cores are balanced (too many cores per executor increases GC pressure) and consider enabling external shuffle service or local SSDs for heavy shuffle workloads. Iterate with targeted runs and monitor shuffle bytes, task skew, and stage durations until the bottleneck is resolved.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

23. You see data skew on a key join. What steps would you take to resolve it?

First I prove it -don't guess. I pull per-key counts and stage-level metrics (task duration, shuffle bytes, bytes read) from the job UI to quantify how skewed the distribution is (e.g., top 0.1% keys = X% of rows). That tells me whether a few "heavy hitters" are the problem or a long tail of moderately-heavy keys. Diagnosis guides the fix: a handful of hot keys need different tactics than wide-but-uneven key distribution.

If one or a few keys dominate, I treat them as special cases. The usual pragmatic approach is salting (shard the heavy key): add a random/round-robin shard suffix to the join key on the large side, perform the join in parallel across shards, then un-shard by aggregating the partial results. This spreads load across executors while preserving correctness; you tune the shard count until tasks balance. For joins where the small side truly is "small," I prefer broadcasting that side to avoid the shuffle altogether.

When skew is caused by many medium-weight keys, I try to reduce the amount of data shuffled first: push filters/projections earlier, use CombinePerKey/map-side pre-aggregation, or re-partition data to a higher parallelism that matches cluster capacity. If the dataset is repeatedly joined on the same key, consider bucketing/partitioning by that key at write time so future jobs reuse physical locality and avoid repeated full shuffles.

If the platform supports adaptive execution (AQE on Spark, runner features in Beam/Dataflow), enable it -AQE can detect skew at runtime and split large partitions dynamically or coalesce tiny ones. For production, I often combine techniques: special-case heavy hitters, enable AQE, and pre-aggregate where possible. Always include monitoring (per-key latency, task skew) and a small POC to validate the chosen shard count or partitioning strategy before rolling it to production.

Finally, add governance: document which keys are handled specially, add an automated skew detector that alerts when top-key dominance crosses a threshold, and include a replay-friendly compaction/repair runbook. That way skew is caught early and your mitigation (salting, broadcast, pre-aggregate, bucketing) is repeatable and auditable.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

24. A pipeline requires repeated access to intermediate data. Would you cache, checkpoint, or both? Why?

Short answer: use caching when you need fast, repeated reads and the intermediate dataset fits (or mostly fits) in memory; use checkpointing when you need to truncate long lineage for fault tolerance or to avoid expensive recomputation overhead -and in many production cases you'll use both together for best effect.

If a stage is iterated over many times (machine-learning training loops, iterative algorithms, or repeated downstream queries), persist/cache that RDD/DataFrame in memory (or memory+disk) with an appropriate storage level (MEMORY_ONLY, MEMORY_AND_DISK). Caching gives low-latency reuse and avoids repeated IO. But be mindful: caching consumes executors' memory and can cause eviction or GC pressure if not sized properly -monitor and unpersist() once the reuse window is finished.

Checkpointing complements caching: if the DAG lineage is long or if recomputing an upstream stage is expensive, checkpointing writes the dataset to reliable storage (HDFS/GCS) and truncates the lineage. That reduces the cost and risk of long lineage recomputation after failures, and it prevents very deep DAGs from blowing driver/executor memory. In Spark, a common pattern is df.persist() then df.checkpoint() (after enabling a reliable checkpoint directory) so you get fast reads plus a stable recovery point.

Use both when the intermediate is reused and the lineage is complex: cache for speed, checkpoint to avoid long recompute on driver restarts or executor loss. For streaming or stateful jobs, checkpointing (or savepoints) is essential for exactly-once recovery, while caching isn't typically used. Always pick an appropriate checkpoint frequency (not every micro-step) to balance I/O cost and recovery guarantees.

Operationally: size your cluster so cached datasets have enough headroom, choose MEMORY_AND_DISK if memory is tight, set checkpoint dirs on durable, low-latency storage, and add metrics/alerts for cached memory pressure and checkpoint lag. Finally, add an automated unpersist/cleanup step in pipelines and a test that simulates node failure so you validate your caching+checkpointing strategy under realistic failure scenarios.

25. You need to run a PySpark job followed by a Hive query automatically. How would you orchestrate this?

I'd use a workflow orchestrator (Cloud Composer / Airflow is my default on GCP) and model the two steps as sequential tasks in a single DAG: first a Dataproc spark job (PySpark) and then a Dataproc/Hive job (HiveQL or beeline) that runs only after the spark task succeeds. Using Dataproc operators (DataprocSubmitJobOperator) keeps things simple and auditable -the operator will submit the job to a short-lived cluster or a shared cluster and return the job id for monitoring and logs. I prefer ephemeral clusters for batch ETLs (spin up → run → tear down) to avoid idle costs, but if many jobs run frequently, point the operator at a pre-warmed cluster and use a sensor to ensure the cluster is healthy first.

Make the Spark job and the Hive query idempotent and produce clear handoff artifacts: the PySpark step should write cleaned/partitioned output to a well-known GCS path or staging BigQuery table and emit a manifest/_SUCCESS marker. The Hive step should read that manifest (or use a Dataproc metastore table pointing at the GCS location) rather than guessing which files to process. That way retries or DAG re-runs don't accidentally reprocess the same data twice and the hive step has a deterministic input to query/LOAD/MERGE against.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

Add robust failure handling and retries in the DAG: give the Spark task a small retry budget with exponential backoff for transient errors, but don't blindly retry on logical data failures -route those to a DLQ and mark the DAG branch for manual remediation. On success/failure emit meaningful metrics and notifications (Cloud Monitoring + Pub/Sub/Slack) and capture job logs (Dataproc logs are accessible in Stackdriver) so operators can triage quickly. If the pipeline touches sensitive data, ensure the service account used has least privilege and any credentials/secrets are stored in Secret Manager and injected at runtime.

For testing and safe deployment, I'd run the full DAG in a dev environment in "shadow" mode where the Hive step runs but writes to a staging table so QA can compare Spark output vs Hive results. Use Airflow templating for parameters (dates/paths), include a "dry-run" flag that validates manifests without committing to production tables, and include automated post-run checks (row counts, key checksums) that gate downstream downstream publishing. Finally, version the DAG + job templates in source control and wire job submissions through CI so any change to the PySpark or Hive logic is peer-reviewed and tested before it touches production.

26. Your workflow must run daily with different input paths. How would you parameterize it?

I'd parameterize the DAG so the input GCS path (or BigQuery partition, or date window) is a runtime argument rather than baked into code. In Airflow/Cloud Composer that means using DAG run configuration or macros: use {{ ds }} or {{ logical_date }} to derive gs://bucket/source/date={{ ds }} automatically for daily runs, and expose optional DAG params (start_date, end_date, force=true) for backfills or manual replays. That keeps a single DAG template reusable and avoids hard-coded paths across environments.

For secure or environment-specific parameters (credentials, bucket names, CMEK key IDs), keep them in Airflow Variables or in Secret Manager and reference them from the DAG at runtime. For larger configuration (mapping of sources → sinks, transformation flags), store a JSON config file in GCS and pass its URI as a DAG param; the job reads the config at start and validates the schema before processing. This pattern makes the job data-driven and lets non-developers launch parameterized runs safely.

Make the parameterization deterministic and idempotent: record the runtime parameters and a generated batch_id in a manifest table (BigQuery) at job start so you can trace exactly which inputs were used for each run. Use that manifest as the canonical lock -the orchestrator should insert a manifest row with status=RUNNING (using an atomic upsert) and only proceed if no conflicting run for the same partition exists. On success update status=SUCCESS with row counts and checksums. That prevents double-processing when someone accidentally kicks off overlapping runs.

Support backfills and ad-hoc runs by exposing CLI or API entrypoints that accept the same parameters (gcloud composer environments run ... --conf '{"date":"2025-09-15"}') so automation and humans use the same contract. Add a validation step at the top of the DAG that checks the input path exists and the _SUCCESS marker (or manifest) is present, and optionally runs a lightweight schema/row-count check. If parameters can change format over time, include unit/integration tests in CI that exercise a variety of parameter combinations so you catch configuration errors before they hit production.

Lastly, document the parameter contract and provide sample invocation templates (daily cron, manual CLI, ad-hoc JSON) plus a small dashboard that shows the last N runs and their input paths. That makes operations straightforward, enables safe ad-hoc reprocessing, and ensures the daily parameterized workflow is auditable and reproducible.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

27. A job fails midway. How would you configure retries in workflow templates?

Start by making retries explicit in the workflow template, not implicit: set retry policies per task (or per operator) with a sensible max attempts, exponential backoff and jitter. For transient errors you want a few quick retries with exponential backoff (e.g., 3–5 attempts, base delay 30s, multiplier 2, max delay few minutes) so temporary service flaps or quota hiccups are handled without human intervention.

Differentiate retry behavior by failure type -don't blindly retry permanent failures. Configure the workflow to inspect error classes (or use task exit codes) and only apply automatic retries for transient categories (network/timeouts/5xx). For known permanent errors (schema mismatch, validation failure) fail fast and route the payload to a dead-letter artifact so operators can triage instead of wasting resources on futile retries.

Use idempotent tasks and safe commit semantics so retries are harmless. That means write side-effects only after a successful checkpoint, or employ staging → finalize patterns (write to staging and MERGE/swap on success). Where tasks are not naturally idempotent, add dedupe tokens or a manifest table so repeated attempts don't double-process data.

Add observability and automation around retries: emit retry counters, oldest retry age, and alert if an execution exceeds retry budget. Also build a retry escalation step in the workflow -after automatic retries are exhausted, automatically snapshot inputs and push metadata to a rerun queue or an operator ticket so human review is streamlined.

Finally, document retry policies in the template and keep them configurable by environment. Use template parameters for max-retries/backoff so dev/staging can be aggressive while prod is conservative; include a graceful shutdown/drain path in your runbook so operators can safely intervene without losing visibility or corrupting downstream data.

28. Your client wants workflows managed in multiple environments. How would you migrate them?

Treat environments as immutable, parameterized deployments. Store workflow definitions, templates, and configs in Git; keep environment-specific values (project IDs, buckets, service accounts, secrets) out of code and in environment-specific config files or secret stores. That way the same workflow artifact is promoted across dev → staging → prod with predictable differences only in parameters.

Automate the promotion pipeline: CI builds the workflow artifact (template, Docker image, SQL migrations), runs integration tests in an isolated test env, and then a CD pipeline promotes the artifact to staging and later to prod -with manual approvals for sensitive stages. Use IaC (Terraform/Deployment Manager) to provision environment scaffolding so networking, IAM, and KMS settings are reproducible and versioned alongside the workflows.

Handle data and state carefully during migration: don't copy live production data into lower envs unless scrubbed. Instead use synthetic or sampled datasets and maintain a consistent metadata/manifest model so tests and smoke runs in staging exercise the same paths as prod. For migrations requiring schema changes, use blue/green or shadow runs: run the new workflow in shadow against the same inputs and compare outputs before switching traffic.

Adopt environment-aware templates and parameterization so a single artifact can run in multiple projects/accounts. Bake in safe defaults (dry-run, write-to-staging flags) and support a --promote or --switch step for cutover. Maintain a central registry of deployed versions per environment so rollback is a single button or script (redeploy previous artifact and drain the new one).

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

Finally, operationalize governance and runbooks: require PRs + review for workflow changes, keep deployment approvals auditable, and provide clear runbooks for operators (how to validate parity, how to replay failing runs). With automation, parameterization, and strong governance, you get predictable, low-risk migrations across multiple environments.

29. You want to push Spark DataFrames directly into BigQuery. How would you configure it?

Use Google's Spark-BigQuery connector -it's the standard, well-supported bridge. Add the connector jar to your Dataproc cluster (or include it in your job's --jars / image). In Spark code you then write with df.write.format("bigquery").option("table","project:dataset.table").save() (or save() with the table option). That hides the heavy lifting and gives you options for load vs streaming semantics.

Decide the write method up front. For large bulk loads the connector's **indirect** mode writes Parquet to a GCS temp location and issues a BigQuery **load job** (cheap and robust). For low-latency writes use the **direct** Storage Write API (writeMethod=direct) but be aware it's more sensitive to retries and quotas. Pick indirect for batch ETL and direct if you need near-real-time and can tolerate the operational complexity.

Configure a GCS temporary bucket in the same region as your BigQuery dataset and give the job's service account roles/storage.objectCreator on that bucket plus roles/bigquery.dataEditor (or narrower) on the target dataset. Supply the temp bucket via .option("temporaryGcsBucket","gs://my-temp-bucket") for indirect loads. Keep the temp bucket lifecycle/cleanup rules in CI/CD so temporary objects don't pile up.

Tune write options: set createDisposition/writeDisposition (CREATE_IF_NEEDED, WRITE_TRUNCATE/WRITE_APPEND), consider partitioning/clustering at the BigQuery side to speed queries, and control parallelism by repartitioning the DataFrame before write so you produce a suitable number of load files (avoid thousands of tiny files). Also enable retries/backoff in your job and log the connector's job ids so you can replay if a load fails.

Finally, validate and monitor: run small end-to-end tests (schema compatibility, nullability), check BigQuery load job statistics in Cloud Console, and add post-load validations (row counts, checksums). If you need CMEK, ensure the dataset and the temp bucket are configured appropriately and the service account has cloudkms.cryptoKeyEncrypterDecrypter on the key - otherwise writes will fail at runtime.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

30. A pipeline requires querying BigQuery tables inside Spark. Which connector would you use?

Use the Google spark-bigquery-connector (the Spark BigQuery connector) -it supports high-throughput reads by leveraging the BigQuery Storage API under the hood and integrates cleanly with Spark DataFrame APIs. Add the connector jar to the cluster and then read with spark.read.format("bigquery").option("table","project:dataset.table").load() to pull data into a DataFrame. This gives parallel, columnar reads and SQL push-down for many filters.

Prefer the Storage API path (the connector does this automatically) for large analytic reads because it streams columnar data efficiently (lower latency and CPU than JDBC). For small ad-hoc queries you could use the BigQuery JDBC driver, but JDBC is slower and less parallel -use it only for tooling or when you need SQL clients that only speak JDBC. The connector + Storage API is the go-to for production ETL.

Tune read performance: push predicates/column lists to Spark so BigQuery can prune partitions and reduce bytes scanned; set parallelism or tune DataFrame partitioning after read if you need specific executor parallelism. If the table is partitioned or clustered, include a WHERE on the partition column to massively cut scanned bytes. Also choose readDataFormat/arrow options if your connector version exposes them to optimize serialization.

Handle credentials and locality: give the job's service account roles/bigquery.dataViewer and (if indirect methods used) roles/storage.objectViewer on any temp GCS bucket. Keep BigQuery and the Dataproc region aligned to avoid cross-region egress costs. For very heavy reads, consider staging extracts to partitioned Parquet on GCS (via a BigQuery export or ELT job) and then read those Parquet files from Spark -that can be cheaper/more repeatable for repeated jobs.

Finally, operationalize: test with representative data to size spark.sql.shuffle.partitions, enable metrics for bytes read and read time, and prefer connector versions that match your Spark/Dataproc image. If you expect schema evolution, make your read/transform code tolerant (nullable columns, struct handling) and include CI tests that validate schema compatibility before running large jobs.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

31. Your job is expensive when writing to BigQuery. How would you reduce cost?

First, identify *why* the cost is high -is it BigQuery load/streaming costs, excessive bytes processed by downstream queries, or repeated full-table rewrites? Once you know the cause, prefer batch file loads over streaming inserts: stage output as compressed Avro/Parquet on GCS and perform a single BigQuery load job. File-based loads are far cheaper per byte than streaming API calls and avoid streaming buffer overhead and per-row costs.

Second, reduce the bytes written and re-written. Only write the columns you actually need (drop wide unused columns), push down filters so you don't write unneeded rows, and compact small files before load (coalesce/repartition appropriately in Spark) to avoid many small-load operations. Use columnar formats (Parquet/Avro) with compression to minimize storage uploaded and make loads faster and cheaper.

Third, design the target table to avoid costly repeated full-table operations: use partitioned and clustered tables so future queries scan less data and merges/updates can be limited to partitions. When you need to update existing rows, write new data to a staging table and use a single MERGE that targets only affected partitions instead of rewriting the whole table.

Fourth, tune connector and job-level settings: prefer bulk-load mode of whatever BigQuery connector you use (staging to GCS + load), avoid auto-retries that duplicate writes without idempotency, and set appropriate writeDispositions (e.g., WRITE_APPEND to append only new partitions rather than WRITE_TRUNCATE which rewrites). Also ensure you're not using unnecessarily high slot reservations or extra compute in BigQuery for the job; if you control slots, match them to actual needs.

Finally, add operational guardrails: add schema/type checks and row-count expectations to detect runaway jobs, schedule large loads off-peak if you have flexible slot pricing, and maintain monitoring and cost-alerting so spikes are visible. Together these steps cut both the data transferred and the expensive BigQuery operations that drive cost.

32. A schema mismatch occurs when writing Spark data to BigQuery. How would you handle it?

Start by validating and normalizing the Spark DataFrame schema before the write. Compare DataFrame column names, types, and nullability against the BigQuery table schema and explicitly cast or transform columns that don't match (e.g., df.withColumn("ts", col("ts").cast("timestamp")), rename columns to expected names, and add missing columns with lit(None).cast(expectedType) so the write schema matches exactly).

If the mismatch is missing columns on BigQuery, prefer writing to a staging table first (same schema as incoming data) and then use a controlled SQL ALTER TABLE ADD COLUMN or a MERGE into the production table after validating data. This staged approach prevents partial failures and makes schema changes explicit and auditable rather than letting the connector silently fail or produce unexpected types.

If the mismatch is caused by data that doesn't conform to target types (e.g., strings in numeric columns), add a validation/cleansing step in Spark: attempt safe casts with fallback rules, surface bad rows to a quarantine dataset, and only write cleaned data to BigQuery. Logging the number and examples of bad rows helps decide whether to fix upstream or relax the target schema.

Use connector-supported schema options carefully rather than relying on automatic inference. Many BigQuery connectors support writing Avro/Parquet which preserves nested and nullable types -prefer those formats to JSON streaming. If you must allow schema evolution, use explicit schema-update operations (e.g., allowing field additions) executed deliberately, not as a side-effect of a write job; automated schema updates can hide problems.

© Shubham Wadekar

Finally, automate schema checks in CI/CD for pipelines: keep canonical BigQuery table schemas in source control, run a lightweight schema-compatibility test before deployment, and fail fast with clear error messages when incompatibilities appear. That reduces surprises in production and gives a repeatable, safe process for handling schema changes.

33. You need to store raw data in GCS and process it in Dataproc. How would you design the pipeline?

Start with a clear landing (raw) zone in GCS with a strict naming & partitioning convention (for example gs://project-raw/{source}/{yyyy}/{MM}/{dd}/file_<ingest_ts>.json). Keep raw files immutable -never overwrite -and include a manifest/metadata file or write object metadata (producer, schema version, checksum, ingest timestamp). Use separate buckets or prefixes for different environments (dev/stage/prod) and apply bucket-level IAM, CMEK encryption and requestor-pays if appropriate.

Ingest: choose the right ingestion pattern. For batch, have producers upload files to the raw bucket (or push them to a staging area and atomically move into raw/ after validation). For streaming, use Pub/Sub + Cloud Storage sink, or write streaming buffers and periodically flush to GCS. Validate at ingest (lightweight checksums, filename schema) and emit ingest audit events to a Pub/Sub topic or logging service.

Orchestration: trigger processing with Cloud Scheduler / Cloud Functions / Workflows or orchestrate via Airflow (Cloud Composer). Use file arrival triggers (Cloud Functions on OBJECT_FINALIZE) only for small/controlled flows -for large volumes prefer time-windowed batch triggers to avoid a thundering herd of jobs.

Dataproc processing: run ephemeral, autoscaled Dataproc clusters (or Dataproc Serverless) to run Spark/Hive jobs that read raw objects from GCS. Best practices:

- Use preemptible workers to lower compute cost and enable autoscaling policies to scale to zero when idle.
- Submit jobs programmatically (Dataproc Jobs API) from the orchestrator and capture job metadata and logs.
- Use initialization actions or containerized jobs to install dependencies and set the correct connectors (e.g., Hadoop GCS connector).
- Read raw files into Spark, perform schema inference/validation, cleansing, and heavy transformations in-memory, and write results in optimized formats.

Output & layout: write processed data to a separate **processed (curated) zone** as partitioned, columnar, compressed files (Parquet/Avro) with partitioning aligned to query patterns (date, customer_id, etc.). If downstream is BigQuery, either stage Parquet on GCS then load into BigQuery (bulk load) or write streams to BigQuery only when necessary. Keep a separate **sandbox/qa** area for intermediate datasets and quarantined bad records.

Metadata/catalog & governance: register datasets with Data Catalog (or Glue-like catalog), record schema versions, provenance (who/when), and job lineage. Maintain a schema registry or keep canonical schemas in source control. Enforce IAM at dataset/prefix level and apply object lifecycle rules (e.g., move raw to Nearline/Coldline after X days, or delete after retention period).

Observability & reliability: emit structured logs and metrics (job start/finish, record counts, bytes processed, errors). Capture data quality checks (row count, null thresholds, schema drift) as job steps and fail early. Add alerting for anomalies and cost spikes. Keep retries idempotent (write to staging locations and promote after successful completion) to avoid duplicate writes.

© Shubham Wadekar

CI/CD & testing: keep Spark jobs and job config in repo, run unit tests for transforms, include schema compatibility checks in CI, and use infra-as-code (Terraform) for buckets, IAM, and Dataproc clusters.

Security & cost controls: use least-privilege service accounts for Dataproc and orchestrators, enable CMEK if required, and apply lifecycle rules and compression/partitioning to control storage and egress costs.

34. Your team complains about high costs due to storing uncompressed CSVs in GCS. What would you suggest?

Switch from uncompressed CSV to a compressed, columnar format as your default for stored and processed data. Concretely:

- Convert CSVs to Parquet (or Avro/ORC) and compress with a cheap, splittable codec (Parquet + Snappy is a common, performant default). Parquet is columnar, so it reduces storage and downstream query scan costs dramatically.
- If you must keep raw text for audit, keep a small window (e.g., 7–30 days) of the original CSVs, then archive or delete them after you've validated and converted them. Use object lifecycle rules to automatically transition older raw CSVs to Nearline/Coldline/Archive storage classes or delete them per retention policy.

Additional practical actions:

- If immediate conversion is expensive, at minimum gzip the CSVs on ingest (e.g., file.csv.gz) -this reduces storage but note gzip files aren't splittable, so for large-scale parallel processing prefer Parquet/Avro.
- Partition and compact files: avoid many small CSVs (per-file object overhead) -coalesce into reasonably sized files (hundreds of MBs) before storing, which improves both storage efficiency and processing performance.
- Remove unnecessary columns and PII before storing raw if business allows -fewer bytes = lower cost.
- Use lifecycle management to move rarely-read data to cheaper storage tiers (Nearline/Coldline) automatically after a defined age.
- Monitor and report: add storage cost dashboards and set alerts for bucket growth so teams understand cost drivers and can act earlier.

Finally, pick a migration plan: run a one-time batch job that reads existing CSVs, writes Parquet to the processed bucket, verifies counts, and then either delete or archive the original CSVs. After that, update producers or ingestion pipelines to write Parquet/Avro directly (or gzip CSV as an interim step). These changes typically cut storage and query costs substantially while improving downstream performance.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

35. Too many small files are slowing down jobs. How would you solve this?

First, fix the root cause (prevent small files) and then remediate existing ones.

Prevention (preferred)

- Batch upstream writes: change producers to write larger files (buffer & flush by size/time) instead of many tiny files.
- Use a streaming-to-batch writer: route events to Pub/Sub or Kafka and have a worker (Dataflow / a small Spark process) aggregate into periodically-written large files (e.g., 100–512 MB).
- Write in a splittable, compressed columnar format (Parquet/Avro with Snappy) rather than many small CSVs; these formats are both storage- and query-efficient.

Remediation (compact existing small files)

- Run a compaction job on Dataproc/Dataproc Serverless / Dataflow:
    - Read the small-file prefix with Spark, coalesce/repartition to a target number of files, write out Parquet/Avro with compression and partitioning aligned to query patterns.
    - Example approach: spark.read.option(...).parquet("gs://bucket/prefix/*").coalesce(N).write.mode("overwrite").parquet("gs://bucket/compacted/").
- Use a rolling/periodic batch compaction (e.g., daily) for new arrivals so compaction is incremental and bounded.
- Avoid gzipped CSV compaction pitfalls: gzip is not splittable -convert to Parquet/Avro during compaction.

GCS-specific helpers & caution

- You can use object composition for simple concatenation of objects in GCS, but it has limitations (doesn't recompress, metadata rules, not suitable for compressed files or producing columnar formats). Prefer reading+rewriting via compute so you can change format and partitioning.

Tuning & sizing guidance

- Target file sizes that balance parallelism and overhead -commonly 128–512 MB for Parquet files (adjust for your cluster & IO pattern).
- Pick number of output files per partition based on partition cardinality and query concurrency. Use repartition() when partitioning by a high-cardinality key, or coalesce() when you only need fewer files.

Operational practices

- Automate compaction as part of ingest workflow (i.e., producers write to a staging prefix, compact job consolidates into final prefix and then atomically swaps or updates pointers).
- Add metrics (files-per-prefix, avg file size) and alerts; enforce policies (reject producers that create >X files/hour) if necessary.
- Make compaction idempotent: write to a staging path and promote on success to avoid duplicates.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

Result: fewer large, well-partitioned, compressed files → faster job startup, fewer RPCs, better IO utilization, and lower overall cost.

36. A pipeline must trigger when new files land in GCS. How would you implement this?

Use GCS event notifications → Pub/Sub → a small, resilient orchestrator (Cloud Function / Cloud Run / Dataflow / Cloud Composer) that validates and starts the processing job. Key design points:

Minimal architecture

1. Enable GCS notifications to a Pub/Sub topic for OBJECT_FINALIZE (new object) events.
2. Subscribe with a Cloud Function or Cloud Run service that receives the Pub/Sub message, validates the object (path, size, file type), and triggers the downstream job (Dataproc job submission, Dataflow job, Cloud Run worker, or push a message to an orchestrator queue).

Handling scale and de-duplication

- Use Pub/Sub as the buffer -it smooths bursts and retries.
- Make the processing idempotent (e.g., write to a staging path with a deterministic name, check for an existing marker file before processing).
- Use attributes (object name, generation/ETag) to detect replays and avoid duplicate processing. Add a short-lived dedupe store (Memorystore / Redis) if needed.

Batching & debouncing (important for many small files)

- If many small files land together, avoid launching one job per file. Options:
    - The Cloud Function can push file metadata into a Pub/Sub batching topic or a datastore table; a scheduled worker (Cloud Scheduler → Cloud Function / Cloud Run) runs every X minutes to process a batch window.
    - Or use Dataflow's streaming pipeline to window events (e.g., 5–10 minute fixed windows) and write a single consolidated file or submit one job per window.

Triggering Dataproc jobs

- From the Cloud Function/Cloud Run, call the Dataproc Jobs API (or use gcloud dataproc jobs submit in automation) passing the GCS path(s) and job parameters. Use service accounts with least privilege and record job metadata for lineage/audit.

Validation, retries & dead-letter

- Validate object size and schema early; if it's malformed, move to a quarantine prefix and push an alert.
- Rely on Pub/Sub delivery + exponential backoff; on repeated failures send the message to a dead-letter topic for manual inspection.

Security & observability

- Least-privilege service accounts for the trigger component and Dataproc.
- Emit structured logs (file received, validation result, job id) and metrics (events/sec, failures, lag). Set alerts for high failure rates or large unprocessed backlog.

Quick example flow (practical)

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

- GCS → Pub/Sub (OBJECT_FINALIZE) → Cloud Function (validates + batches) → Pub/Sub batch topic → Scheduler or Dataflow window → submit Dataproc job reading gs://batch/*.avro → write results to processed/ prefix and write a completion marker.

That design is scalable, debuggable, avoids thundering-herd problems, and supports both immediate and windowed (batched) processing depending on throughput and latency needs.

37. You need to process IoT data streams in near real-time with Dataproc. How would you build it?

I'd treat Dataproc as the execution engine but design the pipeline around Spark Structured Streaming (not legacy DStream) so I get a concise, testable API for windowing, state and event-time handling. Devices publish into Pub/Sub (or Pub/Sub Lite) and Dataproc runs a Spark Structured Streaming job that pulls from that subscription, parses each message into a typed schema (device_id, event_ts, payload, event_id, schema_version) and assigns the event-time timestamp right away. That gives me correct windowing and late-data handling instead of relying on processing time.

Because IoT is often out-of-order and bursty, I'd use event-time windowing with watermarks and withWatermark to bound state retention (choose watermark = typical network jitter + safety margin). For per-device sessions or rolling aggregates I'd use stateful APIs (mapGroupsWithState or flatMapGroupsWithState) or Spark's aggregation with windowing, and keep per-key state compact with TTLs. Hot-key handling (very chatty devices) is solved by sharding the key or offloading extremely-hot device aggregates to a specialized store like Bigtable.

Operational durability is crucial: checkpoint Structured Streaming state and offsets to a durable GCS checkpoint directory so jobs can recover from failure. Persist the raw events immediately to a cold archive in GCS (partitioned by ingestion date) so you have an immutable changelog for forensic replays and backfills. For sinks, write low-latency alerts/commands to Pub/Sub or a Redis/Bigtable layer, and write enriched/audited events or aggregates to BigQuery (partitioned + clustered) for analytics -make those writes idempotent (merge/upsert or use unique event ids) because Spark's guarantees are typically at-least-once.

For scale and cost control, run the streaming Spark job on a long-running Dataproc cluster sized for steady-state load and enable autoscaling if traffic varies, or use an ephemeral cluster pattern only if you can tolerate startup latency. Tune Spark resources (executor memory/cores, parallelism) and use efficient serialization (Kryo) and fast codecs (Snappy) for shuffle/IO. If ultra-low tail latency is required, measure and consider using Dataflow for lower-operational-cost streaming; but Dataproc + Structured Streaming is perfectly fine for second-level latency when configured well.

Finally, add observability and safety: instrument metrics (events/sec, watermark lag, state size per key, DLQ counts), route malformed messages to a DLQ (GCS or Pub/Sub) and provide a replay tool, secure service accounts with least-privilege to Pub/Sub/GCS/BigQuery, and document an operational runbook for draining, rolling upgrades and backfills. That yields a robust, replayable near-real-time IoT pipeline on Dataproc that supports both low-latency needs and authoritative backfills.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

38. A Spark Streaming job must consume Pub/Sub messages. How would you configure it?

I'd use Spark Structured Streaming with the Google Cloud Pub/Sub connector (the maintained connector that reads from a Pub/Sub subscription using the BigQuery Storage-like parallel readers), and configure it as a streaming source: specify the subscription/project, set spark.readStream.format("pubsub") options (subscription or topic), and include the GCP credentials or service-account impersonation the cluster will use. Always set a durable checkpointLocation on GCS so offsets and streaming state survive restarts.

Control ingestion and backpressure explicitly: use maxOffsetsPerTrigger (or equivalent rate-limiting options) and choose an appropriate trigger (processingTime micro-batch interval) so you balance latency with per-batch overhead. If ordering per-device is required, use Pub/Sub ordering keys or key-by device_id in Spark and ensure downstream stateful operators are keyed consistently; for exactly-once semantics, design sinks to be idempotent (BigQuery MERGE or Storage Write API with insertId, Bigtable with deterministic timestamps/versions).

Handle schema parsing and failures defensively: parse the Pub/Sub payload into a typed schema in a map/flatMap stage, validate fields and route malformed records to a dead-letter GCS path or a DLQ topic rather than failing the stream. For late or out-of-order events, use withWatermark and windowing semantics in your aggregations. If you need session or per-key state, use mapGroupsWithState with state timeouts and a compact state representation so state size stays manageable.

Tune cluster and Spark settings for streaming: pick machine types with enough memory to hold state, configure executor cores to avoid oversubscription (too many cores per executor can increase GC), enable Kryo serialization, and set appropriate spark.sql.shuffle.partitions for the expected throughput. Monitor Pub/Sub ack lag, Dataflow (if used) metrics, Spark streaming latency percentiles, and executor memory/GC -then adjust micro-batch interval and parallelism to hit your latency SLA without overwhelming downstream sinks.

Finally, secure and operate it: grant the Dataproc service account roles/pubsub.subscriber on the subscription and roles/storage.objectAdmin (or restricted) on the checkpoint bucket, rotate keys or use Workload Identity, instrument retry metrics, and provide a replay path from the raw GCS archive for debugging or backfills. With that configuration you get a resilient Spark streaming consumer of Pub/Sub suitable for production real-time processing.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

39. You need to push processed streaming results back to Pub/Sub. How would you design it?

First, I'd treat the Pub/Sub write as a first-class sink in the streaming pipeline and design for correct semantics. That means the pipeline (whether Spark Structured Streaming, Dataflow/Beam, or Flink) must reliably produce message payloads and include a stable message ID or key in the attributes so downstream consumers can detect duplicates. For at-least-once sources like Pub/Sub, I'd add idempotency on the consumer side or use deduplication logic downstream -or write a unique message ID (UUID or deterministic key) with each message so subscribers can ignore repeats.

Second, I'd pay attention to throughput, batching and latency trade-offs. Configure the sink to batch messages for efficiency but keep batch sizes and flush intervals tuned for your latency SLA. Use Pub/Sub ordering keys if message order for a key matters, and partition your workload by that key so ordered delivery is preserved. If you need ultra-low latency and high QPS, consider Pub/Sub Lite or tuned publisher client settings (max outstanding messages, batching settings, retry/backoff).

Third, operational robustness is critical: implement retries with exponential backoff on publish failures, surface failures to a dead-letter topic when retries exhaust, and add health checks and metrics -publish successes, publish latency, outstanding messages, retry counts. Ensure proper IAM: a service account with least privilege to publish to the specific topic, and use VPC Service Controls or private Google access if you're in a VPC with tight networking needs.

Fourth, design for state and checkpointing. If using Spark Structured Streaming, enable checkpointing and store offsets/checkpoints in a durable store (GCS, HDFS). That ensures the pipeline can resume without re-publishing already-processed outputs or can at least detect reprocessing. If ordering and exactly-once matter, combine checkpointed state with idempotent publish keys or an external dedup store (e.g., Bigtable) to confirm whether an event was already published.

Finally, include schema and contract thinking up front: use a stable message schema (Avro/Proto/JSON Schema) and version it, add metadata attributes (source job, timestamp, event id), and set up monitoring and alerts (dead-letter growth, publish error rate, end-to-end latency). Run load tests and chaos tests (simulate failures, restarts, network blips) so your retry/backoff, checkpointing and DLQ behavior are validated before production roll-out.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

40. Your client is comparing Spark Streaming with Dataflow. How would you explain which to choose?

I'd start by clarifying the core difference: Spark Structured Streaming is part of the Spark ecosystem and typically runs as micro-batches (though continuous processing exists), while Dataflow implements the Apache Beam model with a cloud-managed, true streaming runtime built for event-time semantics and autoscaling. That shapes trade-offs: Spark is powerful when you have heavy batch+stream convergence and want the Spark libraries; Dataflow is strong when you want a managed, low-ops streaming engine with excellent event-time handling.

Next, consider operational burden and scaling. Dataflow is a fully managed service that autos-scales and handles a lot of streaming operational complexity for you -which reduces DevOps effort and often gives more predictable autoscaling for varying workloads. Spark (on Dataproc or self-managed clusters) gives you more control and can be cheaper at steady large scale, but you'll manage cluster sizing, scaling policies, and upgrades yourself unless you use managed Dataproc autoscaling.

Then evaluate semantics, latency and integrations. If your pipeline requires strict event-time correctness, complex windowing with late-data handling, and low tail latency, Dataflow's Beam model and runner have a maturity advantage and tighter integration with Pub/Sub. Spark Structured Streaming is excellent for micro-batch semantics and heavy transformations, but for the lowest-latency, per-event processing and advanced event-time features Dataflow tends to win.

Also weigh ecosystem and team skills. If your team is invested in Spark (Scala libraries, MLlib, GraphX, existing jobs), sticking with Spark reduces retraining and enables sharing code between batch and streaming. If the team prefers a cloud-native, serverless experience or you want portability via Beam across runners, Dataflow is attractive. Language support differs too: Spark has mature Scala/Java/Python support, Beam has Java/Python (and evolving SDKs).

Finally, make a pragmatic recommendation: choose Dataflow when you want managed, resilient streaming with excellent event-time semantics, minimal ops, and tight GCP integration. Choose Spark when you need advanced Spark-only libraries, already have a Spark ecosystem, or must run across clouds/on-prem. If unsure, prototype a critical pipeline end-to-end on both with representative data -the comparison on cost, latency, and operational overhead for your workload will make the decision clear.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

41. A cluster runs fine during the day but is underutilized at night. How would you optimize it?

Start by measuring and understanding the nightly workload pattern -how much CPU, memory, and I/O are idle, which jobs run (if any), and whether any jobs could be shifted to off-peak. With that visibility you can choose the simplest fix: schedule cluster scale-down at night and scale-up before the morning ramp. If you're on a cloud-managed service (Dataproc, EMR, EKS, etc.) use the provider autoscaler or scheduled instance groups to reduce node count overnight and restore it in the morning.

Next, consider using preemptible/spot instances or lower-cost VM types for non-critical or batch-only workloads that run at night. Move optional/elastic work (e.g., long-running aggregations, re-indexing, nightly ETL) onto a cheaper, preemptible worker pool so you keep a small steady core and use cheap capacity for bursty tasks without wasting full-price nodes.

Also think about workload consolidation and shifting. If there are periodic maintenance tasks, machine learning model training, index compactions, or heavy batch processing that don't need daytime responsiveness, move them to the night window so you use the cluster instead of idling. Conversely, if there truly is no night work, schedule automatic shutdown of the cluster or create ephemeral clusters that spin up only when needed -this is often the biggest cost saver.

Don't forget to right-size and tune configuration: reduce minimum node counts, shrink oversized instance types, tune YARN/container settings or K8s pod resource requests so idle capacity isn't reserved but left usable. Add alerts so autoscaling actions don't impact SLAs; keep a small "buffer" pool for sudden load (fast warm-up) or use warm pool instances to avoid cold-start latency.

Finally, bake this into process: add cost/runbook checks, automated nightly reports, and periodic reviews. If you're in a multi-tenant environment, coordinate with teams to schedule heavy jobs during off-peak and enforce quotas or incentives so the cluster stays efficiently utilized long-term.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

42. A job backlog is growing in YARN queues. How would you handle scaling?

First, diagnose why backlog is growing -is it sudden traffic increase, slower jobs, misconfigured queues, or resource starvation? Check queue metrics (waiting jobs, maxRunning, average runtime), node health, and recent deployments or data size changes. Understanding whether this is demand-driven or a capacity/configuration issue guides whether you scale infra or tune scheduling.

If it's true capacity shortage, scale the cluster horizontally by adding nodes or increasing node sizes so YARN can launch more containers. If you're on cloud, prefer autoscaling groups or a managed autoscaler that reacts to pending container counts. If adding infra isn't an option, consider vertical tuning: increase container sizes, reduce overhead per container (fewer tiny containers), or increase concurrency limits per node safely.

If the backlog is caused by unfair queue configuration or a noisy tenant, tune the Capacity or Fair scheduler: rebalance queue capacities, raise priority for business-critical queues, enable preemption for high-priority work, and set max-running limits for lower-priority users. Enforce or adjust queue ACLs, and introduce or tighten resource quotas so one job or user doesn't starve the rest.

Parallel to scaling, optimize the jobs themselves: profile the heaviest jobs and reduce shuffle, tune memory/GC, cache reused datasets, or rewrite inefficient joins. Sometimes a small code/config change (coalesce, partitioning, caching) increases throughput more than adding hardware. Also implement admission controls -e.g., limit concurrent heavy jobs and enqueue non-urgent work for off-peak processing.

Finally, create a short-term remediation + long-term plan: short-term add nodes or bump capacities; medium-term rebalance scheduler and enforce quotas; long-term implement autoscaling, better job cost accounting, SLAs by queue, and ask teams to optimize heavy jobs. Monitor the queue backlog with alerts so you catch regressions early.

43. You want to cut costs using preemptible VMs. How would you configure them safely?

I'd start by separating my cluster into two clear pools: a small, reliable pool of non-preemptible masters/core nodes that hold critical state and a larger pool of preemptible/spot workers for ephemeral compute. That way a preemption doesn't take down the master or destroy metadata; only cheap workers get reclaimed. On managed services (Dataproc/EMR/AKS) I'd use the built-in secondary/spot worker group or a mixed instance group so the control plane stays stable.

Next, I'd make the work running on preemptible workers resilient to interruptions. For Spark and similar engines that means enabling checkpointing and durable shuffle/storage to GCS/S3 (not local disk), using short-lived tasks where possible, and writing intermediate results frequently. For streaming, pick write-ahead logs or state backends that survive worker loss; for batch, stage shuffle spill and temporary files to a durable store so jobs can recover or be retried without data loss.

Operationally, I'd handle preemptions gracefully: tune client-side batching and retries with exponential backoff, make publishes idempotent, and instrument metrics/alerts specifically for preemption rates and job restarts. Configure autoscaling to replace preempted capacity quickly, and provide a fallback policy -if preemption spikes beyond a threshold, automatically fall back to non-preemptible workers for critical queues to protect SLAs.

Cost-wise, mix instance types and use lower-cost zones/regions only if acceptable, and prefer short-lived preemptible instances for embarrassingly parallel or fault-tolerant workloads (map-heavy, no long-lived shuffle). I'd also use lifecycle policies: prefer preemptible for non-urgent,

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

off-peak or batch windows; tag and account jobs so teams know which workloads are spot-capable and which are not.

Finally, secure and test the setup: give spot workers least-privilege service accounts, ensure logs and job output land in durable, access-controlled buckets, and run production-like chaos tests (simulated preemptions) to validate recovery behavior. Document the trade-offs and set clear SLAs for teams so everyone knows which jobs can use preemptible capacity and which must run on stable nodes.

44. A workload is one-off. How would you design ephemeral clusters for it?

I'd treat the ephemeral cluster as a scripted, single-purpose resource: provision it just-in-time, run the job, persist outputs, and destroy it. That means automating creation with IaC (Terraform/CloudFormation) or a simple job-launch script that creates a minimal control plane, worker nodes sized for the job, attaches IAM roles, runs the job, copies results to durable storage (GCS/S3/BigQuery), and tears down the cluster on success or failure.

To speed startup and reduce variability I'd use pre-baked images or custom machine images with all dependencies installed so boot-time init is quick. If using managed services, enable initialization actions (Dataproc init actions, EMR bootstrap) to install only tiny, job-specific bits. Keep the cluster small and focused: one master, right-sized workers, and no extra services. This reduces cost and reduces the attack surface.

Persisting state and logs is critical because the cluster will be destroyed. I'd write all job outputs, checkpoints, and logs to durable storage before destroying the cluster, and stream logs to a central logging system so debugging after teardown is straightforward. Also export job exit codes and metadata to a central place (job tracker or a DB) so automation can detect success/failure and avoid orphaned invoices.

For reliability, make the job idempotent and include retry logic in the orchestration layer rather than relying on the ephemeral cluster to handle many retries. Use shorter task granularity if cancellation is likely, and set a sensible timeout and failure policy (e.g., on repeated failures, snapshot environment for debugging before teardown). Use an automated hook to create a cost-hold or keep a minimal debug cluster only when needed.

Finally, secure and monitor the process: assign least-privilege credentials to the ephemeral cluster, ensure network access is limited, tag everything for cost attribution, and add post-run cleanup checks. After the job run, capture metrics (runtime, cost, peak resources) so you can right-size the next ephemeral run and keep the approach predictable and repeatable.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

45. You need to build a raw + curated + analytics data lake on GCS. How would Dataproc fit in?

Dataproc's role is primarily as the processing engine that moves data from the raw landing zone into curated, analytics-ready formats. I'd land immutable, time-stamped raw files in GCS (opinionated: a date-partitioned prefix like gs://bucket/raw/<source>/<yyyy>/<mm>/<dd>/) and treat that as the single source of truth. Dataproc (Spark/Hadoop jobs) then reads raw objects from GCS, applies cleansing, enrichment, deduplication and schema validation, and writes partitioned columnar files (Parquet/ORC) into a curated path. The advantage is Dataproc's native support for Spark and its tight GCS connector -jobs can scale, use cluster-local memory for heavy shuffles, and write directly to durable cloud storage.

For streaming or near-real-time portions, Dataproc can be used with structured streaming or by running Flink on Dataproc to transform events before landing into curated buckets, but for continuous streams you may also pair Dataproc with Dataflow. For batch ELT pipelines, Dataproc excels: schedule ephemeral clusters per job to keep costs low, use initialization actions to install libraries (Hudi/Delta/Iceberg) if you need transactional updates, and tear clusters down on completion. Ephemeral Dataproc + autoscaling minimizes cost while giving full Spark power for complex transformations.

Metadata and discoverability are important: register curated datasets in a central metastore (Dataproc Metastore or an external Hive Metastore) and a data catalog (Data Catalog) so analysts and downstream jobs know schema, partitions and lineage. Dataproc jobs should emit lineage metadata (what raw files were read, job run id, schema version) into the catalog or a lightweight metadata store so you can trace curated outputs back to raw inputs for debugging and reprocessing decisions.

For analytics consumption, Dataproc can also prepare and load optimized tables into analytical engines -for example, write partitioned Parquet to the curated layer and then either create external BigQuery tables over that data or load the curated data into native BigQuery tables (batch load or streaming insert). Dataproc can run the transformations and then use the BigQuery connector (or Storage Write API) to push curated results to BigQuery for high-performance SQL analytics, ML, and BI. If you want ACID-like upserts on curated data in GCS, run Spark with Hudi/Iceberg/Delta on Dataproc and use their table metadata for compaction and incremental reads.

Operationally, use CI/CD for Dataproc job artifacts, implement robust monitoring (stackdriver/Cloud Monitoring for job health, cost, preemptions), and secure everything with least-privilege service accounts, object-level IAM, CMEK if needed, and VPC Service Controls for sensitive data. Finally, bake lifecycle rules on raw/cured buckets (archive or cold storage) and enforce partitioning/clustering and compaction policies via Dataproc jobs so query cost and performance remain predictable.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

46. A client wants to query curated data in BigQuery but store raw in GCS. How would you design the flow?

Start with a clear separation of concerns: raw immutable storage in GCS as the ingestion layer, and curated, schema-stable data exposed to analysts inside BigQuery. Ingestion writes raw payloads (JSON/Avro/files) to gs://bucket/raw/... using the source-native producers (streaming via Pub/Sub, direct writes, or batch uploads). Keep the raw layout timestamped and immutable so you can always reprocess deterministically; do not apply normalization or destructive edits in raw.

Next, use a processing layer (Dataproc, Dataflow, or a mix depending on workload) to transform raw into curated. For batch-heavy ETL or complex joins and ML feature engineering use Dataproc/Spark; for continuous stream normalization use Dataflow. The processing job reads raw files, validates and enforces schema, partitions and writes curated Parquet/ORC into gs://bucket/curated/<domain>/<table>/... and also writes a canonical copy into BigQuery. For BigQuery ingestion I prefer either (a) a managed load job that loads partitioned Parquet from GCS to partitioned BigQuery tables (cost-efficient and fast for batch), or (b) the BigQuery Storage Write API for streaming/near-real-time inserts when low latency is needed.

Decide between external tables and native BigQuery tables intentionally. External tables (BigQuery federated to GCS) let you avoid storage duplication but have higher query cost and can be slower for heavy analytic workloads; native BigQuery tables give best query performance and flexible clustering/partitioning options. My default for analytics is to load curated, frequently-queried data into BigQuery native tables, and keep raw and less-frequently used historical data in GCS external tables or long-term archival storage.

Operational and governance pieces finish the design: register schemas and dataset metadata in Data Catalog, enforce schema evolution policies, and implement incremental upsert strategies (use partitioned append for most data; use Hudi/Iceberg on the GCS curated layer + Dataproc if you need ACID-like updates before loading). Add access controls so analysts query curated BigQuery datasets with IAM, while raw GCS data is accessible only to processing jobs and auditors. Also add monitoring: end-to-end latency, load job failures, schema drift alerts, and cost tracking for BigQuery slot usage and GCS egress.

Lastly, automate and test the flow: orchestration (Cloud Composer/Airflow or Workflows) triggers processing when raw files arrive, the pipeline writes curated artifacts and then runs a verified load to BigQuery, and a final QA step validates row counts and checksums. Keep the entire process idempotent -if you re-run an ETL, it should either be a safe upsert or write to a new partition/version -so you can recover from partial failures without data corruption.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

47. How would you use Hive Metastore in Dataproc to manage schema in your data lake?

I'd start by treating the Hive Metastore as the single source of truth for table schemas and partitions across all compute engines that touch the lake -Spark, Hive, Presto/Trino, and any SQL-on-Hadoop tools. On GCP I'd prefer Dataproc Metastore (a managed Hive Metastore) or an HA external Hive Metastore so schema metadata is centrally available, avoids drift, and prevents every job from re-inferring schema from files. That makes discovery, partition pruning, and schema validation consistent across teams.

Operationally I'd register curated datasets in the metastore with explicit table definitions (location, partitioning, column types) rather than relying on implicit schema-on-read. For partition-heavy datasets I'd manage partitions programmatically from ETL jobs (ALTER TABLE ADD PARTITION) or use a partition discovery step at job end so the metastore always reflects what's actually on GCS. This enables query engines to prune partitions and avoid scanning unnecessary data -huge perf and cost wins.

For schema evolution, I'd define a clear policy: additive changes (new nullable columns) are allowed automatically, while destructive or type-changing operations go through a controlled migration process. Implement schema migrations in CI/CD: migration scripts that update the metastore, run compatibility checks against downstream consumers, and include rollback steps. Where transactional semantics are needed, I'd combine Hive Metastore with table formats like Hudi/Iceberg/Delta (managed via Dataproc) so you get metadata, snapshotting and safe upserts plus metastore visibility.

Security and availability matter: put the metastore in a highly available configuration, limit access with IAM and Kerberos where supported, and use service accounts for jobs. Back up metastore metadata regularly and store exports (DDL and partition lists) so you can rebuild metadata if it is corrupted. Also emit and store lineage and audit metadata (who changed schema, when) alongside the metastore to help with debugging and governance.

Finally, integrate the metastore with your catalog and governance stack -sync Dataproc Metastore with Data Catalog or your data catalog solution so analysts can discover datasets, schemas, owners, and tags. Add automated tests that validate job outputs against metastore schemas (row counts, column types) and bake these checks into your deployment pipeline so schema changes don't silently break consumers.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

48. What best practices would you apply when combining GCS as a lake with BigQuery as a warehouse?

I'd enforce a clear separation of layers: immutable raw in GCS, curated/optimized data in GCS (columnar, partitioned) and analytic-ready tables in BigQuery. Raw should be stored as original payloads (JSON/Avro) and never mutated; curated should be Parquet/ORC with sensible partitioning and compaction so both external readers and BigQuery load jobs can operate efficiently. This separation makes reprocessing safe and gives you predictable cost and performance characteristics.

Choose formats and partitioning thoughtfully: write Parquet/Avro with compression and consistent partition keys (date, region, business key). For BigQuery, prefer loading curated data into native partitioned & clustered tables for performance -external BigQuery tables over GCS are fine for ad-hoc or infrequent queries but native tables will usually be faster and cheaper at scale. Also avoid many tiny files: implement compaction as part of ETL (Dataproc jobs or orchestration) so queries don't pay a penalty for metadata overhead.

Optimize data movement and costs: use batch loads from GCS into BigQuery for large volumes (load jobs read compressed Parquet efficiently) and reserve streaming inserts for low-latency needs. Be deliberate about slot and reservation planning: isolate heavy analytical workloads with reservations or use flat-rate billing when predictable. Monitor egress and query bytes - partition pruning, clustering, and predicate pushdown by using well-designed schemas reduces scanned bytes and cost.

Governance and metadata are critical: register tables, schemas and lineage in Data Catalog (or your catalog) so analysts can discover and trust datasets. Enforce access control so BigQuery datasets are accessible to analysts while raw GCS is restricted to processing service accounts and auditors. Apply encryption (CMEK if required), VPC Service Controls for sensitive projects, and audit logging to meet compliance needs.

Finally, automate and test the pipeline: orchestrate ingestion and loads with Airflow/Composer or Workflows, run checksum/row-count validations after loads, and alert on drift or failed loads. Measure and iterate with cost/latency dashboards, and keep a playbook to decide when to use external tables vs native BigQuery, when to compact files, and when to push more compute into Dataproc vs leaving data in BigQuery -these operational rules keep the lake+warehouse pattern efficient and predictable.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

49. You're migrating Hadoop MapReduce jobs to Dataproc. What would you change?

First, I'd replace HDFS-centric thinking with cloud storage -change all HDFS paths to gs:// locations and use the GCS connector. That means updating job inputs/outputs, temporary directories, and any code that assumes local/HDFS file semantics (like file renames or relying on atomic moves). Move intermediate and final data to durable GCS buckets so jobs are resilient to node preemption or cluster tear-down.

Second, rethink job pattern and performance assumptions. MapReduce jobs often expect many small tasks and heavy disk I/O; on Dataproc you get fast networked I/O and different trade-offs. Tune mapper/reducer counts, memory, and container sizes for the Dataproc VM types you'll use. Where MapReduce writes many small files, add a compaction step or change reducers to emit fewer, larger files to avoid metadata overhead on GCS.

Third, adjust operational glue: replace on-prem job schedulers and HDFS-based checkpoints with cloud-native orchestration and monitoring. Migrate Oozie workflows to Cloud Composer / Airflow or use Dataproc workflows; wire logs to Cloud Logging and metrics to Cloud Monitoring. Update retry, checkpointing and failure recovery to rely on durable storage and orchestration retries rather than local node state.

Fourth, update security and dependencies. Move from Kerberos/HDFS ACLs to IAM-based access controls for GCS and configure service accounts with least privilege. Package and stage any custom JARs or native libraries either in GCS or use Dataproc init actions so clusters have required dependencies. Also ensure native binaries (native compression, snappy/zlib) are available or enabled on images.

Finally, evaluate whether to keep MapReduce or refactor: for many teams, a pragmatic migration is "lift-and-shift" MapReduce to Dataproc to get immediate benefits; for longer-term maintainability and performance, consider porting to Spark or Dataflow (fewer lines of code, better in-memory processing). Regardless, add CI/CD validation, smoke tests against sampled data, and run cost and performance comparisons before committing to the final path.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

50. A Spark job in Scala must move to GCP. How would you migrate it to Dataproc?

First, package the Scala job as a self-contained fat JAR (assembly) using sbt-assembly or Maven shade, pinning the Spark version to match Dataproc's runtime. Remove any bundled Hadoop/GCS connector versions that conflict with the cluster runtime and instead rely on the Dataproc image's preinstalled connectors, or stage compatible connector JARs via init actions if needed.

Second, swap filesystem references and I/O patterns to cloud-friendly paths -replace hdfs:// with gs:// for input, output, checkpoints, and temp. Where the job used local disks for shuffle spill or caching, prefer Dataproc defaults (ephemeral local SSDs) or configure Spark to spill to GCS-resilient locations for durability. Also, change any direct ZooKeeper/HDFS metadata operations to cloud equivalents (Dataproc Metastore, GCS metadata, BigQuery).

Third, tune Spark configuration to the cloud environment: set executor/driver memory and cores for the chosen VM types, tune spark.sql.shuffle.partitions to avoid tiny tasks, and use dynamic allocation or Dataproc autoscaling to control cluster size. If you rely on HDFS-specific optimizations (combiner patterns, custom input formats), re-test and adapt them for GCS semantics and network I/O characteristics.

Fourth, wire the job into GCP ops: use gcloud dataproc jobs submit spark or the Dataproc API/Workflows from your CI/CD pipeline to run the job on ephemeral clusters, send logs to Cloud Logging, and export metrics to Cloud Monitoring. If the job writes analytical outputs, consider writing partitioned Parquet to GCS and then loading to BigQuery via load jobs or using the BigQuery connector from Spark for direct writes.

Finally, validate and harden: run end-to-end tests on representative data, add retries and idempotency for any side effects, secure service accounts with least privilege for GCS/BigQuery access, and create post-run validations (row counts, checksums). After a successful pilot, automate cluster creation and job submission, capture cost/performance metrics, and iterate on resource sizing and shuffle/partition tuning for production.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

51. A Hive query runs on on-prem Presto. How would you migrate it to Dataproc?

I'd start by understanding what the query depends on: table definitions, partitions, custom SerDes or UDFs, and where the data lives (HDFS or NFS). That inventory tells me whether I need to move data to GCS, adapt schemas in a metastore, or port custom code. I'd also capture query performance characteristics (scan size, runtime, and skew) so I can validate parity after migration.

Next, I'd move the data to GCS in an organized way -keep the same partitioning and file formats if possible (Parquet/ORC preferred), or convert JSON/CSV to columnar during the copy. While doing this, I'd register the tables in a central Hive Metastore (Dataproc Metastore) so Spark, Presto/Trino on Dataproc, and other engines see the same schema and partitions without re-inferring them from files each time.

Then I'd choose the execution engine on Dataproc that best matches Presto semantics: either run Trino/Presto on Dataproc or run Spark SQL/Presto-on-Dataproc, depending on team skills and SLAs. If I migrate to Spark SQL, I'd validate SQL compatibility (some Presto functions differ) and port UDFs -recompile Java/Scala UDFs or rewrite them in Spark SQL/Python where feasible. For short-term parity, running Presto/Trino on the Dataproc cluster minimizes SQL changes.

After that I'd focus on tuning and testing: adjust executor sizes, shuffle partitions, and file sizes to match cloud VM types; test partition pruning and predicate pushdown; run the captured queries and compare row counts, latencies, and costs against the on-prem baseline. I'd also add CI smoke tests for critical queries, and run a subset of production workload for a validation window to catch edge cases.

Finally, operationalize the new flow: point orchestration (Airflow/Composer) at Dataproc jobs or Presto service on Dataproc, centralize logs to Cloud Logging, and set IAM so analysts access curated datasets in BigQuery or via the metastore. Document any SQL differences, update runbooks for failures, and iterate on cost/performance (e.g., consider externalizing very frequent analytic tables to BigQuery if query latency or concurrency becomes a bottleneck).

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

52. Your company uses Informatica ETL. How would you modernize with Dataproc?

First I'd map Informatica workflows to their logical components -ingestion connectors, transformations (joins, lookups, aggregations), orchestration, and targets -so we can design equivalent building blocks on GCP. This gives a migration plan that preserves business logic while exposing opportunities for cost, scale and maintainability improvements.

Next, I'd replace connectors and heavy-lifting with cloud-native patterns: ingest with Pub/Sub or Cloud Storage for batch, use Dataproc (Spark) for large-scale transformations and joins, and consider Dataflow for streaming or event-time processing. For many Informatica transforms, a Spark job on Dataproc is a direct fit; lookups can become broadcast joins or keyed state depending on size and latency needs.

Then I'd reimplement control flow and scheduling in Airflow/Cloud Composer or Workflows, turning monolithic Informatica processes into modular DAGs with clear retry and idempotency behavior. This also enables better CI/CD: store transformation code in git, build artifacts (JARs or Python packages), and deploy via automated pipelines that submit Dataproc jobs or trigger Dataflow templates.

Operationally I'd add metadata, lineage and governance: register datasets in Dataproc Metastore and Data Catalog, enforce schema checks, and write audit logs for dataset versions. For targets like BigQuery, use the Storage Write API or load jobs from curated Parquet in GCS. I'd also plan a phased migration -start with low-risk pipelines, validate outputs, then migrate critical flows -and implement dual-running (Informatica + cloud) during validation where feasible.

Finally, consider cost and people: use ephemeral Dataproc clusters to control spend, leverage preemptible workers for batch where allowed, and invest in upskilling the Informatica team on Spark/Dataflow and Airflow. Document mappings (Informatica transform -> Spark/Dataflow pattern), create runbooks, and measure improvements in cost, throughput and time-to-change to demonstrate the business value of the modernization.

53. Your client wants to cut Dataproc costs. How would you leverage ephemeral clusters?

I'd start by moving from long-lived clusters to ephemeral, job-scoped clusters that are created just-in-time for each batch or ad-hoc job and torn down immediately after success. That eliminates idle hours and ensures you pay only for the compute used during actual processing. Use infrastructure-as-code or Dataproc Workflows / gcloud dataproc commands in CI/CD or Airflow so cluster creation, job submission and teardown are fully automated and repeatable.

Next, right-size the ephemeral cluster templates: pick machine types that match job profiles, set sensible minimum/maximum worker counts, and enable autoscaling or dynamic allocation so the cluster grows only when shuffle or parallelism requires it. Combine this with preemptible/spot workers for the bulk of capacity -keep a small stable core of non-preemptible nodes for job stability and use preemptibles for cheap parallel work, with retry/backoff logic in the job submission layer.

Third, optimize startup time and dependency installation to reduce wasted billable minutes. Use pre-baked custom images or initialization actions that install libraries once, and snapshot warm images for the common Dataproc version so bootstrap is fast. Also design jobs to reuse cached artifacts on GCS or artifact registries rather than rebuilding everything at cluster boot, so most of the runtime is useful work instead of setup.

Fourth, make jobs idempotent and break big monoliths into smaller tasks when practical so failure or preemption only reruns a small piece -this reduces recompute and cost. Persist intermediate state and checkpoints to durable GCS paths and use cooperative task

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

granularities (shorter tasks) so preemptions don't force huge reprocessing windows. Add post-run validations (row counts, checksums) so you avoid costly silent failures that require re-runs.

Finally, add governance and telemetry: tag clusters and jobs for cost attribution, capture runtime/cost metrics (per-job CPU hours, preemptions, startup overhead), and set quotas or policies that prevent runaway cluster creation. Use these metrics to iterate -e.g., if many jobs are tiny and each creates a cluster, consider batching them into one ephemeral cluster run or a lightweight serverless option instead. The combination of automation, right-sizing, preemptibles, and observability delivers the biggest cost wins.

54. You need to design a Dataproc pipeline that runs daily at scale. What would you consider?

First, I'd design for reliable orchestration and idempotency: schedule the pipeline with Airflow/Cloud Composer or Workflows so dependencies, retries and alerts are explicit. Each DAG task should be idempotent -writing outputs to partitioned paths or using atomic rename patterns to avoid partial commits -so reruns (manual or automatic) don't create duplicates or corrupt data. Include pre- and post-run checks (schema, row counts) to validate success before downstream consumers read the data.

Second, plan capacity and autoscaling for predictable daily peaks: use ephemeral Dataproc clusters with autoscaling policies that react to pending containers or job backlog, size worker types to match peak shuffle/memory demands, and prefer preemptible workers for non-critical parallel stages. If job concurrency is high, consider horizontal partitioning of work (per-tenant or per-day shards) so many smaller jobs run in parallel without one huge shuffle-bound job bottlenecking the cluster.

Third, data layout and formats matter: write intermediate and final outputs as partitioned, compressed columnar files (Parquet/ORC) with consistent partitioning keys to enable pruning and reduce downstream query costs. Enforce compaction as part of the pipeline so you don't accumulate many tiny files, and register tables/partitions in a metastore or Data Catalog at job end so query engines can discover new partitions quickly.

Fourth, build robust operational controls: emit detailed metrics (runtime, shuffle bytes, GC pauses, preemption events), wire logs to Cloud Logging, and create alerts for SLA breaches and failure patterns. Implement graceful failure handling -e.g., automatic cluster tear-down on success, snapshot/debug cluster creation on repeated failures, and dead-letter storage for malformed records so you can triage without blocking the whole DAG.

Finally, secure and maintainability considerations: use least-privilege service accounts, store secrets in Secret Manager, maintain job artifacts in a versioned artifact registry, and put CI/CD in place to test jobs on sampled data before promoting. Document SLAs, cost expectations (expected CPU hours), and rollback procedures. With these pieces -orchestration, autoscaling, efficient formats, observability and security -the daily Dataproc pipeline will be scalable, cost-efficient and reliable.

Start by separating concerns: let the orchestration layer (Airflow/Composer, Workflows) handle coarse-grained retries and backoff for failed jobs, and let the processing engine (Spark, Dataflow, Flink) handle fine-grained checkpointing and state recovery. In practice I make each DAG task idempotent so the orchestrator can safely retry a failed step without creating duplicates -for example write outputs to a staging partition and atomically rename into place only after all checks pass.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

For retries use exponential backoff with jitter and a capped retry-count. Don't blindly retry side-effecting actions (external API calls, downstream publishes) without idempotency -instead attach deterministic ids or use transactional/at-least-once-safe sinks (or a dedup store). For transient infra errors, retries in the orchestration layer combined with cluster autoscaling or ephemeral cluster re-creation usually recovers jobs quickly without manual intervention.

Checkpointing must be durable and external to the compute nodes: store offsets, state snapshots and write-ahead logs on durable storage (GCS, Cloud Storage, or a managed state backend). For Spark Structured Streaming enable checkpoint directories on GCS and use checkpoint + write-ahead logs or a transaction-capable sink (Hudi/Iceberg/Hive metastore) if you need stronger guarantees. For Beam/Dataflow rely on the runner's native checkpointing and windowing semantics to handle event-time correctness.

If you need exactly-once semantics for end-to-end workflows, combine engine-level checkpointing with idempotent writes or transactional sinks. Examples: use BigQuery streaming with dedup keys, use Storage Write API with idempotency keys, or write to Hudi/Iceberg tables on GCS and expose a single committed snapshot. Where exactly-once isn't feasible, document at-least-once behavior and build downstream consumers to deduplicate using event IDs or vector clocks.

Finally, operationalize failure handling: emit and monitor checkpoint health, retry counts, lag metrics and dead-letter growth; keep a dead-letter queue for poisoned records with alerts so you can triage without restarting whole pipelines; and run chaos/abort testing so you've validated that retries, checkpoint restores and replays behave as expected before production. Keep runbooks that explain when to let automation retry vs when to escalate to an engineer.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

56. Why is it recommended to separate compute (Dataproc) from storage (GCS) in real-world projects

The primary reason is durability and lifecycle independence: GCS is durable, highly available, and cheap for long-term storage, while Dataproc clusters are ephemeral compute that you spin up and tear down. By keeping raw and curated data in GCS you can destroy and recreate compute clusters without risking data loss or long recovery times, which simplifies operations and reduces cost because you don't pay for always-on nodes just to hold data.

Separation also enables better scalability and elasticity. Storage scales independently and can serve many compute engines (Spark on Dataproc, Dataflow, Trino/Presto, BigQuery external tables) without moving data. That lets you choose the best compute for each task, autoscale clusters up/down for bursty processing, and reuse the same data for analytics, machine learning and ad-hoc queries without costly copies.

From a cost and optimization view, it reduces waste: you avoid paying for idle compute that's only holding data, you can use preemptible workers for processing while keeping data safe, and you can tier data cheaply in GCS (nearline, coldline) as it ages. It also simplifies data governance and lifecycle policies -retention, encryption, and access control live on the object store and are independent of transient compute instances.

Operationally it improves portability and fault tolerance. If a cluster fails or you migrate cloud regions, the data remains intact and accessible; you can replay jobs from the same raw snapshot. It also makes CI/CD and reproducible runs easier -pipelines read the same GCS locations across environments, and Dataproc clusters are just disposable workers that execute transformations against persistent inputs and write durable outputs.

Finally, it enforces a clean architecture and collaboration model: analysts, data engineers and ML teams can discover and consume curated artifacts from the lake without needing cluster access, metadata and catalogs can be centralized, and access controls are simpler to manage at the storage layer. That decoupling leads to clearer ownership, easier auditing, and a more maintainable platform overall.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar